

Ambit – A FEniCS-based cardiovascular multi-physics solver

Dr.-Ing. Marc Hirschvogel

August 14, 2026

Contents

1	Preface	2
2	Installation	3
3	Ambit input	4
4	Physics Models	6
4.1	Solid mechanics	6
	Strong form	6
	Weak form	7
4.2	Fluid mechanics	10
	Eulerian reference frame	10
	ALE reference frame	13
4.3	0D flow: Lumped parameter models	16
	2-element Windkessel	16
	4-element Windkessel (inertance parallel to impedance)	16
	4-element Windkessel (inertance serial to impedance)	16
	In-outflow CRL link	16
	Systemic and pulmonary circulation	16
	Systemic and pulmonary circulation, including capillary flow	20
	Systemic and pulmonary circulation, including capillary and coronary flow	21
	Systemic and pulmonary circulation, capillary flow, and respiratory (gas transport + dissociation) model	23
4.4	Multi-physics coupling	23
	Solid + 0D flow	23
	Fluid + 0D flow	24
	ALE fluid + 0D flow	25
	Fluid-Solid Interaction (FSI)	26
	Fluid-Solid Interaction (FSI) + 0D flow	28
	Fluid-reduced-Solid Interaction (FrSI)	29
	Multiphase Fluid	30
5	Demos	32
5.1	Demo: Solid	32
5.2	Demo: Fluid	33
5.3	Demo: 0D flow	34

5.4	Demo: Solid + 0D flow	37
5.5	Demo: Fluid + 0D flow	39
5.6	Demo: FSI	41
5.7	Demo: Multiphase fluid	46
6	Table of symbols	51

1 Preface

Ambit [11] is an open-source multi-physics finite element solver written in Python, supporting solid and fluid mechanics, fluid-structure interaction (FSI), and lumped-parameter models. It is tailored towards solving problems in cardiac mechanics, but may also be used for more general nonlinear finite element analysis. The code encompasses re-implementations and generalizations of methods developed by the author for his PhD thesis [10] and beyond. Ambit makes use of the open-source finite element library FEniCS/dolfinx (<https://fenicsproject.org>) [16] along with the linear algebra package PETSc (<https://petsc.org>) [2], hence guaranteeing a state-of-the-art finite element and linear algebra backend. It is constantly updated to ensure compatibility with a recent dolfinx development version. I/O routines are designed such that the user only needs to provide input files that define parameters through Python dictionaries, hence no programming or in-depth knowledge of any library-specific syntax is required.

Ambit provides general nonlinear (compressible or incompressible) finite strain solid dynamics [14], implementing a range of hyperelastic, viscous, and active material models. Specifically, the well-known anisotropic Holzapfel-Ogden [15] and Guccione models [9] for structural description of the myocardium are provided, along with a bunch of other models. It further implements strain- and stress-mediated volumetric growth models [8] that allow to model (maladaptive) ventricular shape and size changes. Inverse mechanics approaches to imprint loads into a reference state are implemented using the so-called prestressing method [6] in displacement formulation [18].

Furthermore, fluid dynamics in terms of incompressible Navier-Stokes/Stokes equations – either in Eulerian or Arbitrary Lagrangian-Eulerian (ALE) reference frames – are implemented. Taylor-Hood elements or equal-order approximations with SUPG/PSPG stabilization [20] can be used. Multiphase fluid systems governed by the coupled Navier-Stokes Cahn-Hilliard equations in mass-averaged velocity formulation are also supported [3],[19], with straightforward extension to ALE formulations.

A variety of reduced 0D lumped models targeted at blood circulation modeling are implemented, including 3- and 4-element Windkessel models [22] as well as closed-loop full circulation [13] and coronary flow models [1].

Monolithic fluid-solid interaction (FSI) [17] in ALE formulation using a Lagrange multiplier

field is supported, along with coupling of 3D and 0D models (solid or fluid with 0D lumped circulation systems) such that cardiovascular simulations with realistic boundary conditions can be performed.

Implementations for a recently proposed novel physics- and projection-based model reduction for FSI, denoted as fluid-reduced-solid interaction (FrSI) [12], are provided, along with POD-based Galerkin model reduction techniques [5] using full or boundary subspaces.

The nonlinear (single- or multi-field) problems are solved with a customized Newton solver with PTC [7] adaptivity in case of divergence, providing robustness for numerically challenging problems. Linear solvers and preconditioners can be chosen from the PETSc repertoire, and specific block preconditioners are made available for coupled problems. Avenues for future functionality include cardiac electrophysiology, scalar transport, or finite strain plasticity.

In the following, a brief description of the supported problem types is given, including the strong and weak form of the underlying equations as well as the discrete assembled systems that are solved.

Examples of input files for the respective problem types can be found in the folder **demos** (with detailed setup descriptions) or amongst the test cases in the folder **tests**.

This documentation is structured as follows. In sec. 2, instructions on how to install and use Ambit are given. The relevant supported physics models are described in sec. 4. Demos are presented in sec. 5.

2 Installation

In order to use Ambit, you need to install FEniCSx (<https://github.com/FEniCS/dolfinx#installation>) (latest Ambit-compatible tested dolfinx release version: v0.11.0.post0).

Ambit can then be installed using pip, either the current release

```
python3 -m pip install ambit-fe
```

or latest development version:

```
python3 -m pip install git+https://github.com/marchirschvogel/ambit.git
```

Alternatively, you can pull a pre-built Docker image with FEniCSx and Ambit installed:

```
docker pull ghcr.io/marchirschvogel/ambit:latest
```

If a Docker image for development is desired, the following image contains all dependencies needed to install and run Ambit:

```
docker pull ghcr.io/marchirschvogel/ambit:devenv
```

3 Ambit input

Here, a minimal Ambit input file is shown, exemplarily for a single-field problem. The mandatory parameter dictionaries to provide are input parameters (IO), global control parameters (CTR), time-integration parameters (TME), solver parameters (SOL), finite element parameters (FEM), and constitutive/material parameters (MAT). For multi-physics problems, each field needs individual time, finite element, and constitutive parameters.

```
#!/usr/bin/env python3

# Minimal input file for an elastodynamics problem

import ambit_fe
import numpy as np

def main():

    # Input/output
    IO = {"problem_type"          : "solid",                # type of
        ↪ physics to solve
        "mesh_domain"            : "/path/mesh_d.xdmf",      # path to
        ↪ domain mesh
        "mesh_boundary"          : "/path/mesh_b.xdmf",      # path to
        ↪ boundary mesh
        "meshfile_type"          : "HDF5",                  # encoding
        ↪ (HDF5 or ASCII)
        "write_results_every"    : 1,                      # step
        ↪ frequency for output
        "output_path"            : "/path/...",              # path to
        ↪ output to
        "results_to_write"       : ["displacement",
        ↪ "vonmises_cauchystress"], # results to
        ↪ output
        "simname"                : "my_results_name"}        # midfix of
        ↪ output name

    # Global control
    CTR = {"maxtime"              : 1.0, # maximum simulation time
        "dt"                      : 0.01} # time step size

    # Time discretization
    TME = {"timint"               : "genalpha", # time integration:
        ↪ Generalized-alpha
```

```

    "rho_inf_genalpha" : 1.0}          # spectral radius of
    ↪ Gen-alpha scheme

# Solver
SOL = {"solve_type"      : "direct", # direct linear solver
      "tol_res"          : 1.0e-8,  # residual tolerance
      "tol_inc"          : 1.0e-8}   # increment tolerance

# Finite element discretization
FEM = {"order_disp"      : 1, # FEM degree for displacement field
      "quad_degree"      : 5} # quadrature scheme degree

# Time curves
class TC:
    # user defined load curves, to be used in boundary conditions (BC)
    def tc1(self, t):
        load_max = 5.0
        return load_max * np.sin(t)

# Materials
MAT = {"MAT1" : {"neohooke_dev" : {"mu" : 10.0},          # isochoric
    ↪ NeoHookean material
        "ogden_vol"      : {"kappa" : 1.0e3}, # volumetric
    ↪ Ogden material
        "inertia"        : {"rho0" : 1.0e-6}}} # density

# Boundary conditions
BC = {"dirichlet" : [{"id" : [<SURF_IDs>], # list of surfaces for
    ↪ Dirichlet BC
        "dir" : "all",          # all directions
        "val" : 0.0}],          # set to zero
      "neumann"   : [{"id" : [<SURF_IDs>], # list of surfaces for
    ↪ Neumann BC
        "dir" : "xyz_ref",      # in cartesian reference
    ↪ directions
        "curve" : [1,0,0]]}] # load in x-direction
    ↪ controlled by curve #1 (see time curves)

# Problem setup
problem = ambit_fe.ambit_main.Ambit(io_params=IO, ctrl_params=CTR,
    ↪ time_params=[TME], solver_params=SOL, fem_params=[FEM],
    ↪ constitutive_params=[MAT], boundary_conditions=[BC],
    ↪ time_curves=TC)

```

```

# Run: solve the problem
problem.solve_problem()

if __name__ == "__main__":

    main()

```

4 Physics Models

4.1 Solid mechanics

- Example: Sec. 5.1 and demos/solid
- `problem_type` : "solid"
- Solid mechanics are formulated in a Total Lagrangian frame

Strong form

Displacement-based

- Primary variable: displacement $\mathbf{u}$

$$\begin{aligned}
 \nabla_0 \cdot \mathbf{P}(\mathbf{u}, \mathbf{v}(\mathbf{u})) + \hat{\mathbf{b}}_0 &= \rho_0 \mathbf{a}(\mathbf{u}) & \text{in } \Omega_0 \times [0, T], \\
 \mathbf{u} &= \hat{\mathbf{u}} & \text{on } \Gamma_0^D \times [0, T], \\
 \mathbf{t}_0 = \mathbf{P}\mathbf{n}_0 &= \hat{\mathbf{t}}_0 & \text{on } \Gamma_0^N \times [0, T], \\
 \mathbf{u}(\mathbf{x}_0, 0) &= \hat{\mathbf{u}}_0(\mathbf{x}_0) & \text{in } \Omega_0, \\
 \mathbf{v}(\mathbf{x}_0, 0) &= \hat{\mathbf{v}}_0(\mathbf{x}_0) & \text{in } \Omega_0,
 \end{aligned} \tag{1}$$

Incompressible mechanics

- Primary variables: displacement $\mathbf{u}$ and pressure p

$$\begin{aligned}
 \nabla_0 \cdot \mathbf{P}(\mathbf{u}, p, \mathbf{v}(\mathbf{u})) + \hat{\mathbf{b}}_0 &= \rho_0 \mathbf{a}(\mathbf{u}) & \text{in } \Omega_0 \times [0, T], \\
 J(\mathbf{u}) - 1 &= 0 & \text{in } \Omega_0 \times [0, T], \\
 \mathbf{u} &= \hat{\mathbf{u}} & \text{on } \Gamma_0^D \times [0, T], \\
 \mathbf{t}_0 = \mathbf{P}\mathbf{n}_0 &= \hat{\mathbf{t}}_0 & \text{on } \Gamma_0^N \times [0, T], \\
 \mathbf{u}(\mathbf{x}_0, 0) &= \hat{\mathbf{u}}_0(\mathbf{x}_0) & \text{in } \Omega_0, \\
 \mathbf{v}(\mathbf{x}_0, 0) &= \hat{\mathbf{v}}_0(\mathbf{x}_0) & \text{in } \Omega_0,
 \end{aligned} \tag{2}$$

Weak form

Displacement-based

Find $\mathbf{u} \in \mathcal{V}_u^{h,D}$ such that

$$r(\mathbf{u}; \delta \mathbf{u}) := \delta \mathcal{W}_{\text{kin}}(\mathbf{u}; \delta \mathbf{u}) + \delta \mathcal{W}_{\text{int}}(\mathbf{u}; \delta \mathbf{u}) - \delta \mathcal{W}_{\text{ext}}(\mathbf{u}; \delta \mathbf{u}) = 0, \quad (3)$$

for all $\delta \mathbf{u} \in \mathcal{V}_u^h$

– Kinetic virtual work:

$$\delta \mathcal{W}_{\text{kin}}(\mathbf{u}; \delta \mathbf{u}) = \int_{\Omega_0} \rho_0 \mathbf{a}(\mathbf{u}) \cdot \delta \mathbf{u} \, dV \quad (4)$$

– Internal virtual work:

$$\delta \mathcal{W}_{\text{int}}(\mathbf{u}; \delta \mathbf{u}) = \int_{\Omega_0} \mathbf{P}(\mathbf{u}, \mathbf{v}(\mathbf{u})) : \nabla_0 \delta \mathbf{u} \, dV = \int_{\Omega_0} \mathbf{S}(\mathbf{u}, \mathbf{v}(\mathbf{u})) : \frac{1}{2} \delta \mathbf{C}(\mathbf{u}) \, dV \quad (5)$$

– External virtual work:

- conservative Neumann load:

$$\delta \mathcal{W}_{\text{ext}}(\delta \mathbf{u}) = \int_{\Gamma_0^N} \hat{\mathbf{t}}_0(t) \cdot \delta \mathbf{u} \, dA \quad (6)$$

- Neumann pressure load in current normal direction:

$$\delta \mathcal{W}_{\text{ext}}(\mathbf{u}; \delta \mathbf{u}) = - \int_{\Gamma_0^N} \hat{p}(t) J \mathbf{F}^{-T} \mathbf{n}_0 \cdot \delta \mathbf{u} \, dA \quad (7)$$

- general Neumann load in current direction:

$$\delta \mathcal{W}_{\text{ext}}(\mathbf{u}; \delta \mathbf{u}) = \int_{\Gamma_0} J \mathbf{F}^{-T} \hat{\mathbf{t}}_0(t) \cdot \delta \mathbf{u} \, dA \quad (8)$$

- body force:

$$\delta \mathcal{W}_{\text{ext}}(\delta \mathbf{u}) = \int_{\Omega_0} \hat{\mathbf{b}}_0(t) \cdot \delta \mathbf{u} \, dV \quad (9)$$

- generalized Robin condition:

$$\delta \mathcal{W}_{\text{ext}}(\mathbf{u}; \delta \mathbf{u}) = - \int_{\Gamma_0^R} [k \mathbf{u} + c \mathbf{v}(\mathbf{u})] \cdot \delta \mathbf{u} \, dA \quad (10)$$

- generalized Robin condition in reference surface normal direction:

$$\delta \mathcal{W}_{\text{ext}}(\mathbf{u}; \delta \mathbf{u}) = - \int_{\Gamma_0^{\text{R}}} (\mathbf{n}_0 \otimes \mathbf{n}_0) [k \mathbf{u} + c \mathbf{v}(\mathbf{u})] \cdot \delta \mathbf{u} \, dA \quad (11)$$

Incompressible mechanics: 2-field displacement and pressure variables

Find $(\mathbf{u}, p) \in \mathcal{V}_u^{h,D} \times \mathcal{V}_p^{h,D}$ such that

$$\begin{aligned} r_u(\mathbf{u}, p; \delta \mathbf{u}) &:= \delta \mathcal{W}_{\text{kin}}(\mathbf{u}; \delta \mathbf{u}) + \delta \mathcal{W}_{\text{int}}(\mathbf{u}, p; \delta \mathbf{u}) - \delta \mathcal{W}_{\text{ext}}(\mathbf{u}; \delta \mathbf{u}) = 0, \\ r_p(\mathbf{u}; \delta p) &:= \delta \mathcal{W}_{\text{pres}}(\mathbf{u}; \delta p) = 0, \end{aligned} \quad (12)$$

for all $(\delta \mathbf{u}, \delta p) \in \mathcal{V}_u^h \times \mathcal{V}_p^h$

- Kinetic virtual work: (4)
- Internal virtual work:

$$\delta \mathcal{W}_{\text{int}}(\mathbf{u}, p; \delta \mathbf{u}) = \int_{\Omega_0} \mathbf{P}(\mathbf{u}, p, \mathbf{v}(\mathbf{u})) : \nabla_0 \delta \mathbf{u} \, dV = \int_{\Omega_0} \mathbf{S}(\mathbf{u}, p, \mathbf{v}(\mathbf{u})) : \frac{1}{2} \delta \mathbf{C}(\mathbf{u}) \, dV \quad (13)$$

- Pressure virtual work:

$$\delta \mathcal{W}_{\text{pres}}(\mathbf{u}; \delta p) = \int_{\Omega_0} (J(\mathbf{u}) - 1) \delta p \, dV \quad (14)$$

Material models

- hyperelastic material models

$$\mathbf{S} = 2 \frac{\partial \Psi}{\partial \mathbf{C}} \quad (15)$$

- MAT "neohooke_dev"

$$\Psi = \frac{\mu}{2} (\bar{I}_C - 3) \quad (16)$$

- MAT "holzapfelogden_dev"

$$\Psi = \frac{a_0}{2b_0} \left(e^{b_0(\bar{I}_C - 3)} - 1 \right) + \sum_{i \in \{f, s\}} \frac{a_i}{2b_i} \left(e^{b_i(I_{4,i} - 1)^2} - 1 \right) + \frac{a_{fs}}{2b_{fs}} \left(e^{b_{fs}I_8^2} - 1 \right), \quad (17)$$

$$I_{4,f} = \mathbf{f}_0 \cdot \mathbf{C} \mathbf{f}_0, \quad I_{4,s} = \mathbf{s}_0 \cdot \mathbf{C} \mathbf{s}_0, \quad I_8 = \mathbf{f}_0 \cdot \mathbf{C} \mathbf{s}_0$$

- viscous material models

$$\mathbf{S} = 2 \frac{\partial \Psi_v}{\partial \mathbf{C}} \quad (18)$$

Time integration

– time scheme `timint` : "static"

$$\delta\mathcal{W}_{\text{int}}(\mathbf{u}_{n+1}; \delta\mathbf{u}) - \delta\mathcal{W}_{\text{ext}}(\mathbf{u}_{n+1}; \delta\mathbf{u}) = 0, \quad \forall \delta\mathbf{u} \quad (19)$$

– Generalized-alpha time scheme `timint` : "genalpha"

$$\begin{aligned} \mathbf{v}_{n+1} &= \frac{\gamma}{\beta\Delta t}(\mathbf{u}_{n+1} - \mathbf{u}_n) - \frac{\gamma - \beta}{\beta}\mathbf{v}_n - \frac{\gamma - 2\beta}{2\beta}\Delta t \mathbf{a}_n \\ \mathbf{a}_{n+1} &= \frac{1}{\beta\Delta t^2}(\mathbf{u}_{n+1} - \mathbf{u}_n) - \frac{1}{\beta\Delta t}\mathbf{v}_n - \frac{1 - 2\beta}{2\beta}\mathbf{a}_n \end{aligned} \quad (20)$$

- option `eval_nonlin_terms` : "midpoint":

$$\begin{aligned} \mathbf{u}_{n+1-\alpha_f} &= (1 - \alpha_f)\mathbf{u}_{n+1} + \alpha_f\mathbf{v}_n \\ \mathbf{v}_{n+1-\alpha_f} &= (1 - \alpha_f)\mathbf{v}_{n+1} + \alpha_f\mathbf{v}_n \\ \mathbf{a}_{n+1-\alpha_m} &= (1 - \alpha_m)\mathbf{a}_{n+1} + \alpha_m\mathbf{a}_n \end{aligned} \quad (21)$$

$$\delta\mathcal{W}_{\text{kin}}(\mathbf{a}_{n+1-\alpha_m}; \delta\mathbf{u}) + \delta\mathcal{W}_{\text{int}}(\mathbf{u}_{n+1-\alpha_f}; \delta\mathbf{u}) - \delta\mathcal{W}_{\text{ext}}(\mathbf{u}_{n+1-\alpha_f}; \delta\mathbf{u}) = 0, \quad \forall \delta\mathbf{u} \quad (22)$$

- option `eval_nonlin_terms` : "trapezoidal":

$$\begin{aligned} (1 - \alpha_m) \delta\mathcal{W}_{\text{kin}}(\mathbf{a}_{n+1}; \delta\mathbf{u}) + \alpha_m \delta\mathcal{W}_{\text{kin}}(\mathbf{a}_n; \delta\mathbf{u}) + \\ (1 - \alpha_f) \delta\mathcal{W}_{\text{int}}(\mathbf{u}_{n+1}; \delta\mathbf{u}) + \alpha_f \delta\mathcal{W}_{\text{int}}(\mathbf{u}_n; \delta\mathbf{u}) - \\ (1 - \alpha_f) \delta\mathcal{W}_{\text{ext}}(\mathbf{u}_{n+1}; \delta\mathbf{u}) - \alpha_f \delta\mathcal{W}_{\text{ext}}(\mathbf{u}_n; \delta\mathbf{u}) = 0, \quad \forall \delta\mathbf{u} \end{aligned} \quad (23)$$

– One-Step-theta time scheme `timint` : "ost"

$$\begin{aligned} \mathbf{v}_{n+1} &= \frac{1}{\theta\Delta t}(\mathbf{u}_{n+1} - \mathbf{u}_n) - \frac{1 - \theta}{\theta}\mathbf{v}_n \\ \mathbf{a}_{n+1} &= \frac{1}{\theta^2\Delta t^2}(\mathbf{u}_{n+1} - \mathbf{u}_n) - \frac{1}{\theta^2\Delta t}\mathbf{v}_n - \frac{1 - \theta}{\theta}\mathbf{a}_n \end{aligned} \quad (24)$$

- option `eval_nonlin_terms` : "midpoint":

$$\begin{aligned} \mathbf{u}_{n+\theta} &= \theta\mathbf{u}_{n+1} + (1 - \theta)\mathbf{u}_n \\ \mathbf{v}_{n+\theta} &= \theta\mathbf{v}_{n+1} + (1 - \theta)\mathbf{v}_n \\ \mathbf{a}_{n+\theta} &= \theta\mathbf{a}_{n+1} + (1 - \theta)\mathbf{a}_n \end{aligned} \quad (25)$$

$$\delta\mathcal{W}_{\text{kin}}(\mathbf{a}_{n+\theta}; \delta\mathbf{u}) + \delta\mathcal{W}_{\text{int}}(\mathbf{u}_{n+\theta}; \delta\mathbf{u}) - \delta\mathcal{W}_{\text{ext}}(\mathbf{u}_{n+\theta}; \delta\mathbf{u}) = 0, \quad \forall \delta\mathbf{u} \quad (26)$$

- option `eval_nonlin_terms` : "trapezoidal":

$$\begin{aligned} \theta \delta\mathcal{W}_{\text{kin}}(\mathbf{a}_{n+1}; \delta\mathbf{u}) + (1 - \theta) \delta\mathcal{W}_{\text{kin}}(\mathbf{a}_n; \delta\mathbf{u}) + \\ \theta \delta\mathcal{W}_{\text{int}}(\mathbf{u}_{n+1}; \delta\mathbf{u}) + (1 - \theta) \delta\mathcal{W}_{\text{int}}(\mathbf{u}_n; \delta\mathbf{u}) - \\ \theta \delta\mathcal{W}_{\text{ext}}(\mathbf{u}_{n+1}; \delta\mathbf{u}) - (1 - \theta) \delta\mathcal{W}_{\text{ext}}(\mathbf{u}_n; \delta\mathbf{u}) = 0, \quad \forall \delta\mathbf{u} \end{aligned} \quad (27)$$

Note the equivalence of "midpoint" and "trapezoidal" for all linear terms, e.g. $\delta\mathcal{W}_{\text{kin}}$, or for no or only linear dependence of $\delta\mathcal{W}_{\text{ext}}$ on the solution.

Note that, for incompressible mechanics, the pressure kinematic constraint is always evaluated at t_{n+1} :

$$\delta\mathcal{W}_{\text{pres}}(\mathbf{u}_{n+1}; \delta p) = 0, \quad \forall \delta p, \quad (28)$$

and the pressure in $\delta\mathcal{W}_{\text{int}}$ is set according to (21) or (25), respectively.

Spatial discretization and solution

- Discrete nonlinear system to solve in each time step n (displacement-based):

$$\mathbf{r}_u(\mathbf{u})|_{n+1} = \mathbf{0} \quad (29)$$

- Discrete linear system to solve in each Newton iteration k (displacement-based):

$$\mathbf{K}_{uu}|_{n+1}^k \Delta \mathbf{u}_{n+1}^{k+1} = -\mathbf{r}_u|_{n+1}^k \quad (30)$$

- Discrete nonlinear system to solve in each time step n (incompressible):

$$\mathbf{r}_{n+1} = \begin{bmatrix} \mathbf{r}_u(\mathbf{u}, \mathbf{p}) \\ \mathbf{r}_p(\mathbf{u}) \end{bmatrix}_{n+1} = \mathbf{0} \quad (31)$$

- Discrete linear system to solve in each Newton iteration k (incompressible):

$$\begin{bmatrix} \mathbf{K}_{uu} & \mathbf{K}_{up} \\ \mathbf{K}_{pu} & \mathbf{0} \end{bmatrix}_{n+1}^k \begin{bmatrix} \Delta \mathbf{u} \\ \Delta \mathbf{p} \end{bmatrix}_{n+1}^{k+1} = - \begin{bmatrix} \mathbf{r}_u \\ \mathbf{r}_p \end{bmatrix}_{n+1}^k \quad (32)$$

4.2 Fluid mechanics

Eulerian reference frame

- Example: Sec. 5.2 and `demos/fluid`
- Problem type: `fluid`
- Incompressible Navier-Stokes equations in Eulerian reference frame

Strong Form

– Primary variables: velocity $\mathbf{v}$ and pressure p

$$\begin{aligned}
\rho \left(\frac{\partial \mathbf{v}}{\partial t} + (\nabla \mathbf{v}) \mathbf{v} \right) &= \nabla \cdot \boldsymbol{\sigma}(\mathbf{v}, p) + \hat{\mathbf{b}} && \text{in } \Omega_t \times [0, T], \\
\nabla \cdot \mathbf{v} &= 0 && \text{in } \Omega_t \times [0, T], \\
\mathbf{v} &= \hat{\mathbf{v}} && \text{on } \Gamma_t^D \times [0, T], \\
\mathbf{t} = \boldsymbol{\sigma} \mathbf{n} &= \hat{\mathbf{t}} && \text{on } \Gamma_t^N \times [0, T], \\
\mathbf{v}(\mathbf{x}, 0) &= \hat{\mathbf{v}}_0(\mathbf{x}) && \text{in } \Omega_t,
\end{aligned} \tag{33}$$

with a Newtonian fluid constitutive law

$$\boldsymbol{\sigma} = -p\mathbf{I} + \eta (\nabla \mathbf{v} + (\nabla \mathbf{v})^T) \tag{34}$$

where η is the dynamic viscosity

Weak Form

Find $(\mathbf{v}, p) \in \mathcal{V}_v^{h,D} \times \mathcal{V}_p^{h,D}$ such that

$$\begin{aligned}
r_v(\mathbf{v}, p; \delta \mathbf{v}) &:= \delta \mathcal{P}_{\text{kin}}(\mathbf{v}; \delta \mathbf{v}) + \delta \mathcal{P}_{\text{int}}(\mathbf{v}, p; \delta \mathbf{v}) - \delta \mathcal{P}_{\text{ext}}(\mathbf{v}; \delta \mathbf{v}) = 0, \\
r_p(\mathbf{v}; \delta p) &:= \delta \mathcal{P}_{\text{pres}}(\mathbf{v}; \delta p) = 0,
\end{aligned} \tag{35}$$

for all $(\delta \mathbf{v}, \delta p) \in \mathcal{V}_v^h \times \mathcal{V}_p^h$

– Kinetic virtual power:

$$\delta \mathcal{P}_{\text{kin}}(\mathbf{v}; \delta \mathbf{v}) = \int_{\Omega_t} \rho \left(\frac{\partial \mathbf{v}}{\partial t} + (\nabla \mathbf{v}) \mathbf{v} \right) \cdot \delta \mathbf{v} \, dv \tag{36}$$

– Internal virtual power:

$$\delta \mathcal{P}_{\text{int}}(\mathbf{v}, p; \delta \mathbf{v}) = \int_{\Omega_t} \boldsymbol{\sigma}(\mathbf{v}, p) : \nabla \delta \mathbf{v} \, dv \tag{37}$$

– Pressure virtual power:

$$\delta \mathcal{P}_{\text{pres}}(\mathbf{v}; \delta p) = \int_{\Omega_t} (\nabla \cdot \mathbf{v}) \delta p \, dv \tag{38}$$

– External virtual power:

- conservative Neumann load:

$$\delta \mathcal{P}_{\text{ext}}(\delta \mathbf{v}) = \int_{\Gamma_t^N} \hat{\mathbf{t}}(t) \cdot \delta \mathbf{v} \, da \tag{39}$$

- pressure Neumann load:

$$\delta\mathcal{P}_{\text{ext}}(\delta\mathbf{v}) = - \int_{\Gamma_t^N} \hat{p}(t) \mathbf{n} \cdot \delta\mathbf{v} \, da \quad (40)$$

- body force:

$$\delta\mathcal{P}_{\text{ext}}(\delta\mathbf{v}) = \int_{\Omega_t} \hat{\mathbf{b}}(t) \cdot \delta\mathbf{v} \, dV \quad (41)$$

Stabilization

Streamline-upwind Petrov-Galerkin/pressure-stabilizing Petrov-Galerkin (SUPG/PSPG) methods are implemented, either using the full or a reduced scheme

– to the fluid FEM params, add the dict entry:

```
"stabilization" : {"scheme" : <SCHEME>, "vscale" : 1e3, "dscales" :  
  ↪  [<d1>,<d2>,<d3>],  
          "symmetric" : False, "reduced_scheme" : False}
```

Full scheme according to [20]: "supg_pspg":

– Velocity residual operator in (35) is augmented by the following terms:

$$\begin{aligned} r_v \leftarrow r_v + \frac{1}{\rho} \int_{\Omega_t} \tau_{\text{SUPG}} (\nabla \delta\mathbf{v}) \cdot \left(\rho \left(\frac{\partial \mathbf{v}}{\partial t} + (\nabla \mathbf{v}) \mathbf{v} \right) - \nabla \cdot \boldsymbol{\sigma}(\mathbf{v}, p) \right) dv \\ + \int_{\Omega_t} \tau_{\text{LSIC}} \rho (\nabla \cdot \delta\mathbf{v}) (\nabla \cdot \mathbf{v}) dv \end{aligned} \quad (42)$$

– Pressure residual operator in (35) is augmented by the following terms:

$$r_p \leftarrow r_p + \frac{1}{\rho} \int_{\Omega_t} \tau_{\text{PSPG}} (\nabla \delta p) \cdot \left(\rho \left(\frac{\partial \mathbf{v}}{\partial t} + (\nabla \mathbf{v}) \mathbf{v} \right) - \nabla \cdot \boldsymbol{\sigma}(\mathbf{v}, p) \right) dv \quad (43)$$

– Discrete nonlinear system to solve in each time step n :

$$\mathbf{r}_{n+1} = \begin{bmatrix} \mathbf{r}_v(\mathbf{v}, \mathbf{p}) \\ \mathbf{r}_p(\mathbf{p}, \mathbf{v}) \end{bmatrix}_{n+1} = \mathbf{0} \quad (44)$$

– Discrete linear system to solve in each Newton iteration k :

$$\begin{bmatrix} \mathbf{K}_{vv} & \mathbf{K}_{vp} \\ \mathbf{K}_{pv} & \mathbf{K}_{pp} \end{bmatrix}_{n+1}^k \begin{bmatrix} \Delta \mathbf{v} \\ \Delta \mathbf{p} \end{bmatrix}_{n+1}^{k+1} = - \begin{bmatrix} \mathbf{r}_v \\ \mathbf{r}_p \end{bmatrix}_{n+1}^k \quad (45)$$

– Note that $\mathbf{K}_{pp}$ is zero for Taylor-Hood elements (without stabilization)

ALE reference frame

- Problem type: `fluid_ale`
- Incompressible Navier-Stokes equations in Arbitrary Lagrangian Eulerian (ALE) reference frame
- ALE domain problem deformation governed by linear-elastic, nonlinear hyperelastic solid, or a diffusion problem, displacement field $\mathbf{d}$
- Fluid mechanics formulated with respect to the reference frame, using ALE deformation gradient $\widehat{\mathbf{F}}(\mathbf{d}) = \mathbf{I} + \nabla_0 \mathbf{d}$ and its determinant, $\widehat{J}(\mathbf{d}) = \det \widehat{\mathbf{F}}(\mathbf{d})$

ALE problem

- Primary variable: domain displacement $\mathbf{d}$
- Strong form:

$$\begin{aligned} \nabla_0 \cdot \boldsymbol{\sigma}^G(\mathbf{d}) &= \mathbf{0} \quad \text{in } \Omega_0, \\ \mathbf{d} &= \widehat{\mathbf{d}} \quad \text{on } \Gamma_0^D, \end{aligned} \tag{46}$$

- ALE material `linelast`:

$$\boldsymbol{\sigma}^G(\mathbf{d}) = 2\mu \boldsymbol{\varepsilon} + \lambda \operatorname{tr} \boldsymbol{\varepsilon} \mathbf{I}, \quad \text{with} \quad \boldsymbol{\varepsilon} = \frac{1}{2} (\nabla_0 \mathbf{d} + (\nabla_0 \mathbf{d})^T) \tag{47}$$

- ALE material `diffusion`:

$$\boldsymbol{\sigma}^G(\mathbf{d}) = D \nabla_0 \mathbf{d} \tag{48}$$

- ALE material `neohooke` (fully nonlinear model):

$$\boldsymbol{\sigma}^G(\mathbf{d}) = \frac{\partial \Psi}{\partial \widehat{\mathbf{F}}}, \quad \text{with} \quad \Psi = \frac{\mu}{2} \left(\operatorname{tr}(\widehat{\mathbf{F}}^T \widehat{\mathbf{F}}) - 3 \right) + \frac{\mu}{2\beta} \left(\widehat{J}^{-2\beta} - 1 \right) \tag{49}$$

- weak form:

$$r_d(\mathbf{d}; \delta \mathbf{d}) := \int_{\Omega_0} \boldsymbol{\sigma}^G(\mathbf{d}) : \nabla_0 \delta \mathbf{d} \, dV = 0, \quad \forall \delta \mathbf{d} \tag{50}$$

Strong form (ALE)

– Primary variables: velocity $\mathbf{v}$, pressure p , and domain displacement $\mathbf{d}$

$$\begin{aligned}
\widehat{J}\rho \left(\frac{\partial \mathbf{v}}{\partial t} \Big|_{\mathbf{x}_0} + (\nabla_0 \mathbf{v} \widehat{\mathbf{F}}^{-1}) (\mathbf{v} - \widehat{\mathbf{w}}) \right) &= \nabla_0 \cdot \left(\widehat{J} \boldsymbol{\sigma} \widehat{\mathbf{F}}^{-\text{T}} \right) + \widehat{J} \widehat{\mathbf{b}} \quad \text{in } \Omega_0 \times [0, T], \\
\nabla_0 \cdot \left(\widehat{J} \widehat{\mathbf{F}}^{-1} \mathbf{v} \right) &= 0 \quad \text{in } \Omega_0 \times [0, T], \\
\mathbf{v} &= \widehat{\mathbf{v}} \quad \text{on } \Gamma_0^{\text{D}} \times [0, T], \\
\mathbf{t} = \boldsymbol{\sigma} \mathbf{n} &= \widehat{\mathbf{t}} \quad \text{on } \Gamma_0^{\text{N}} \times [0, T], \\
\mathbf{v}(\mathbf{x}, 0) &= \widehat{\mathbf{v}}_0(\mathbf{x}) \quad \text{in } \Omega_0,
\end{aligned} \tag{51}$$

with a Newtonian fluid constitutive law

$$\boldsymbol{\sigma} = -p \mathbf{I} + \mu \left(\nabla_0 \mathbf{v} \widehat{\mathbf{F}}^{-1} + \widehat{\mathbf{F}}^{-\text{T}} (\nabla_0 \mathbf{v})^{\text{T}} \right) \tag{52}$$

Weak form (ALE)

Find $(\mathbf{v}, p, \mathbf{d}) \in \mathcal{V}_v^{h,D} \times \mathcal{V}_p^{h,D} \times \mathcal{V}_d^{h,D}$ such that

$$\begin{aligned}
r_v(\mathbf{v}, p, \mathbf{d}; \delta \mathbf{v}) &:= \delta \mathcal{P}_{\text{kin}}(\mathbf{v}, \mathbf{d}; \delta \mathbf{v}) + \delta \mathcal{P}_{\text{int}}(\mathbf{v}, p, \mathbf{d}; \delta \mathbf{v}) - \delta \mathcal{P}_{\text{ext}}(\mathbf{v}, \mathbf{d}; \delta \mathbf{v}) = 0, \\
r_p(\mathbf{v}, \mathbf{d}; \delta p) &:= \delta \mathcal{P}_{\text{pres}}(\mathbf{v}, \mathbf{d}; \delta p) = 0, \\
r_d(\mathbf{d}; \delta \mathbf{d}) &= 0,
\end{aligned} \tag{53}$$

for all $(\delta \mathbf{v}, \delta p, \delta \mathbf{d}) \in \mathcal{V}_v^h \times \mathcal{V}_p^h \times \mathcal{V}_d^h$

– Kinetic virtual power:

$$\delta \mathcal{P}_{\text{kin}}(\mathbf{v}, \mathbf{d}; \delta \mathbf{v}) = \int_{\Omega_0} \widehat{J} \rho \left(\frac{\partial \mathbf{v}}{\partial t} \Big|_{\mathbf{x}_0} + (\nabla_0 \mathbf{v} \widehat{\mathbf{F}}^{-1}) (\mathbf{v} - \widehat{\mathbf{w}}) \right) \cdot \delta \mathbf{v} \, dV \tag{54}$$

– Internal virtual power:

$$\delta \mathcal{P}_{\text{int}}(\mathbf{v}, p, \mathbf{d}; \delta \mathbf{v}) = \int_{\Omega_0} \widehat{J} \boldsymbol{\sigma} \widehat{\mathbf{F}}^{-\text{T}} : \nabla_0 \delta \mathbf{v} \, dV \tag{55}$$

– Pressure virtual power:

$$\delta \mathcal{P}_{\text{pres}}(\mathbf{v}, \mathbf{d}; \delta p) = \int_{\Omega_0} \nabla_0 \cdot \left(\widehat{J} \widehat{\mathbf{F}}^{-1} \mathbf{v} \right) \delta p \, dV \tag{56}$$

– External virtual power:

- conservative Neumann load:

$$\delta \mathcal{P}_{\text{ext}}(\delta \mathbf{v}) = \int_{\Gamma_0^{\text{N}}} \widehat{\mathbf{t}}(t) \cdot \delta \mathbf{v} \, dA \tag{57}$$

- pressure Neumann load:

$$\delta \mathcal{P}_{\text{ext}}(\mathbf{d}; \delta \mathbf{v}) = - \int_{\Gamma_0^N} \hat{p}(t) \hat{J} \hat{\mathbf{F}}^{-T} \mathbf{n}_0 \cdot \delta \mathbf{v} \, dA \quad (58)$$

- body force:

$$\delta \mathcal{P}_{\text{ext}}(\mathbf{d}; \delta \mathbf{v}) = \int_{\Omega_0} \hat{J} \hat{\mathbf{b}}(t) \cdot \delta \mathbf{v} \, dV \quad (59)$$

Stabilization (ALE)

"supg_pspg":

- Velocity residual operator in (53) is augmented by the following terms:

$$\begin{aligned} r_v \leftarrow r_v + \frac{1}{\rho} \int_{\Omega_0} \hat{J} \tau_{\text{SUPG}} (\nabla_0 \delta \mathbf{v} \hat{\mathbf{F}}^{-1}) \mathbf{v} \cdot \\ \cdot \left(\hat{J} \rho \left(\frac{\partial \mathbf{v}}{\partial t} \Big|_{\mathbf{x}_0} + (\nabla_0 \mathbf{v} \hat{\mathbf{F}}^{-1}) (\mathbf{v} - \hat{\mathbf{w}}) \right) - \nabla_0 \cdot (\hat{J} \boldsymbol{\sigma} \hat{\mathbf{F}}^{-T}) \right) dV \\ + \int_{\Omega_0} \hat{J} \tau_{\text{LSIC}} \rho (\nabla_0 \delta \mathbf{v} : \hat{\mathbf{F}}^{-T}) (\nabla_0 \mathbf{v} : \hat{\mathbf{F}}^{-T}) dV \end{aligned} \quad (60)$$

- Pressure residual operator in (53) is augmented by the following terms:

$$\begin{aligned} r_p \leftarrow r_p + \frac{1}{\rho} \int_{\Omega_0} \hat{J} \tau_{\text{PSPG}} (\hat{\mathbf{F}}^{-T} \nabla_0 \delta p) \cdot \\ \cdot \left(\hat{J} \rho \left(\frac{\partial \mathbf{v}}{\partial t} \Big|_{\mathbf{x}_0} + (\nabla_0 \mathbf{v} \hat{\mathbf{F}}^{-1}) (\mathbf{v} - \hat{\mathbf{w}}) \right) - \nabla_0 \cdot (\hat{J} \boldsymbol{\sigma} \hat{\mathbf{F}}^{-T}) \right) dV \end{aligned} \quad (61)$$

- Discrete nonlinear system to solve in each time step n :

$$\mathbf{r}_{n+1} = \begin{bmatrix} \mathbf{r}_v(\mathbf{v}, \mathbf{p}, \mathbf{d}) \\ \mathbf{r}_p(\mathbf{p}, \mathbf{v}, \mathbf{d}) \\ \mathbf{r}_d(\mathbf{d}) \end{bmatrix}_{n+1} = \mathbf{0} \quad (62)$$

- Discrete linear system to solve in each Newton iteration k :

$$\begin{bmatrix} \mathbf{K}_{vv} & \mathbf{K}_{vp} & \mathbf{K}_{vd} \\ \mathbf{K}_{pv} & \mathbf{K}_{pp} & \mathbf{K}_{pd} \\ \mathbf{K}_{dv} & \mathbf{0} & \mathbf{K}_{dd} \end{bmatrix}_{n+1}^k \begin{bmatrix} \Delta \mathbf{v} \\ \Delta \mathbf{p} \\ \Delta \mathbf{d} \end{bmatrix}_{n+1}^{k+1} = - \begin{bmatrix} \mathbf{r}_v \\ \mathbf{r}_p \\ \mathbf{r}_d \end{bmatrix}_{n+1}^k \quad (63)$$

- note that $\mathbf{K}_{pp}$ is zero for Taylor-Hood elements (without stabilization)

4.3 0D flow: Lumped parameter models

- Example: Sec. 5.3 and `demos/flow0d`
- Problem type: `flow0d`
- 0D model concentrated elements are resistances (R), impedances (Z , technically are resistances), compliances (C), inertances (L or I), and elastances (E)
- 0D variables are pressures (p), fluxes (q or Q), or volumes (V)

2-element Windkessel

- Model type : `2elwindkessel`

4-element Windkessel (inertance parallel to impedance)

- Model type : `4elwindkesselLpZ`

4-element Windkessel (inertance serial to impedance)

- Model type : `4elwindkesselLsZ`

In-outflow CRL link

- Model type : `CRLinoutlink`

Systemic and pulmonary circulation

- Model type : `syspul`
- Allows to link in a coronary flow model

left heart and systemic circulation

$$\begin{aligned}
-Q_{\text{at}}^\ell &= \sum_{i=1}^{n_{\text{ven}}^{\text{pul}}} q_{\text{ven},i}^{\text{pul}} - q_{\text{v},\text{in}}^\ell && \text{left atrium flow balance} \\
q_{\text{v},\text{in}}^\ell &= q_{\text{mv}}(p_{\text{at}}^\ell - p_{\text{v}}^\ell) && \text{mitral valve momentum} \\
-Q_{\text{v}}^\ell &= q_{\text{v},\text{in}}^\ell - q_{\text{v},\text{out}}^\ell && \text{left ventricle flow balance} \\
q_{\text{v},\text{out}}^\ell &= q_{\text{av}}(p_{\text{v}}^\ell - p_{\text{ar}}^{\text{sys}}) && \text{aortic valve momentum} \\
-Q_{\text{aort}}^{\text{sys}} &= q_{\text{v},\text{out}}^\ell - q_{\text{ar},\text{p}}^{\text{sys}} - \mathbb{I}^{\text{cor}} \sum_{i \in \{\ell, r\}} q_{\text{cor},\text{p},\text{in}}^{\text{sys},i} && \text{aortic root flow balance} \\
I_{\text{ar}}^{\text{sys}} \frac{dq_{\text{ar},\text{p}}^{\text{sys}}}{dt} + Z_{\text{ar}}^{\text{sys}} q_{\text{ar},\text{p}}^{\text{sys}} &= p_{\text{ar}}^{\text{sys}} - p_{\text{ar},\text{d}}^{\text{sys}} && \text{aortic root inertia} \\
C_{\text{ar}}^{\text{sys}} \frac{dp_{\text{ar},\text{d}}^{\text{sys}}}{dt} &= q_{\text{ar},\text{p}}^{\text{sys}} - q_{\text{ar}}^{\text{sys}} && \text{systemic arterial flow balance} \\
L_{\text{ar}}^{\text{sys}} \frac{dq_{\text{ar}}^{\text{sys}}}{dt} + R_{\text{ar}}^{\text{sys}} q_{\text{ar}}^{\text{sys}} &= p_{\text{ar},\text{d}}^{\text{sys}} - p_{\text{ven}}^{\text{sys}} && \text{systemic arterial momentum} \\
C_{\text{ven}}^{\text{sys}} \frac{dp_{\text{ven}}^{\text{sys}}}{dt} &= q_{\text{ar}}^{\text{sys}} - \sum_{i=1}^{n_{\text{ven}}^{\text{sys}}} q_{\text{ven},i}^{\text{sys}} && \text{systemic venous flow balance} \\
L_{\text{ven},i}^{\text{sys}} \frac{dq_{\text{ven},i}^{\text{sys}}}{dt} + R_{\text{ven},i}^{\text{sys}} q_{\text{ven},i}^{\text{sys}} &= p_{\text{ven}}^{\text{sys}} - p_{\text{at},i}^r && \text{systemic venous momentum} \\
&& i \in \{1, \dots, n_{\text{ven}}^{\text{sys}}\}
\end{aligned}$$

right heart and pulmonary circulation

$$\begin{aligned}
-Q_{\text{at}}^r &= \sum_{i=1}^{n_{\text{ven}}^{\text{sys}}} q_{\text{ven},i}^{\text{sys}} + \mathbb{I}^{\text{cor}} q_{\text{cor},\text{d},\text{out}}^{\text{sys}} - q_{\text{v},\text{in}}^r && \text{right atrium flow balance} \\
q_{\text{v},\text{in}}^r &= q_{\text{tv}}(p_{\text{at}}^r - p_{\text{v}}^r) && \text{tricuspid valve momentum} \\
-Q_{\text{v}}^r &= q_{\text{v},\text{in}}^r - q_{\text{v},\text{out}}^r && \text{right ventricle flow balance} \\
q_{\text{v},\text{out}}^r &= q_{\text{pv}}(p_{\text{v}}^r - p_{\text{ar}}^{\text{pul}}) && \text{pulmonary valve momentum} \\
C_{\text{ar}}^{\text{pul}} \frac{dp_{\text{ar}}^{\text{pul}}}{dt} &= q_{\text{v},\text{out}}^r - q_{\text{ar}}^{\text{pul}} && \text{pulmonary arterial flow balance} \\
L_{\text{ar}}^{\text{pul}} \frac{dq_{\text{ar}}^{\text{pul}}}{dt} + R_{\text{ar}}^{\text{pul}} q_{\text{ar}}^{\text{pul}} &= p_{\text{ar}}^{\text{pul}} - p_{\text{ven}}^{\text{pul}} && \text{pulmonary arterial momentum} \\
C_{\text{ven}}^{\text{pul}} \frac{dp_{\text{ven}}^{\text{pul}}}{dt} &= q_{\text{ar}}^{\text{pul}} - \sum_{i=1}^{n_{\text{ven}}^{\text{pul}}} q_{\text{ven},i}^{\text{pul}} && \text{pulmonary venous flow balance} \\
L_{\text{ven},i}^{\text{pul}} \frac{dq_{\text{ven},i}^{\text{pul}}}{dt} + R_{\text{ven},i}^{\text{pul}} q_{\text{ven},i}^{\text{pul}} &= p_{\text{ven}}^{\text{pul}} - p_{\text{at},i}^\ell && \text{pulmonary venous momentum} \\
&& i \in \{1, \dots, n_{\text{ven}}^{\text{pul}}\}
\end{aligned}$$

with:

$$Q_{\text{at}}^\ell := -\frac{dV_{\text{at}}^\ell}{dt}, \quad Q_{\text{v}}^\ell := -\frac{dV_{\text{v}}^\ell}{dt}, \quad Q_{\text{at}}^r := -\frac{dV_{\text{at}}^r}{dt}, \quad Q_{\text{v}}^r := -\frac{dV_{\text{v}}^r}{dt}, \quad Q_{\text{aort}}^{\text{sys}} := -\frac{dV_{\text{aort}}^{\text{sys}}}{dt} \quad (64)$$

and:

$$\mathbb{I}^{\text{cor}} = \begin{cases} 1, & \text{if CORONARYMODEL,} \\ 0, & \text{else} \end{cases} \quad (65)$$

The volume V of the heart chambers (0D) is modeled by the volume-pressure relationship

$$V(t) = \frac{p}{E(t)} + V_{\text{u}}, \quad (66)$$

with the unstressed volume V_{u} and the time-varying elastance

$$E(t) = (E_{\text{max}} - E_{\text{min}}) \cdot \hat{y}(t) + E_{\text{min}}, \quad (67)$$

where E_{max} and E_{min} denote the maximum and minimum elastance, respectively. The normalized activation function $\hat{y}(t)$ is input by the user.

Flow-pressure relations for the four valves, eq. (??), (??), (??), (??), are functions of the pressure difference $p - p_{\text{open}}$ across the valve. The following valve models can be defined:

Valve model `pmlin_pres`:

$$q(p - p_{\text{open}}) = \frac{p - p_{\text{open}}}{\tilde{R}}, \quad \text{with } \tilde{R} = \begin{cases} R_{\text{max}}, & p < p_{\text{open}} \\ R_{\text{min}}, & p \geq p_{\text{open}} \end{cases} \quad (68)$$

Remark: Non-smooth flow-pressure relationship

Valve model `pmlin_time`:

$$q(p - p_{\text{open}}) = \frac{p - p_{\text{open}}}{\tilde{R}}, \quad \text{with } \tilde{R} = \begin{cases} \begin{cases} R_{\text{max}}, & t < t_{\text{open}} \text{ and } t \geq t_{\text{close}} \\ R_{\text{min}}, & t \geq t_{\text{open}} \text{ or } t < t_{\text{close}} \end{cases}, & t_{\text{open}} > t_{\text{close}} \\ \begin{cases} R_{\text{max}}, & t < t_{\text{open}} \text{ or } t \geq t_{\text{close}} \\ R_{\text{min}}, & t \geq t_{\text{open}} \text{ and } t < t_{\text{close}} \end{cases}, & \text{else} \end{cases} \quad (69)$$

Remark: Non-smooth flow-pressure relationship with resistance only dependent on timings, not the pressure difference!

Valve model `smooth_pres_resistance`:

$$q(p - p_{\text{open}}) = \frac{p - p_{\text{open}}}{\tilde{R}}, \quad \text{with } \tilde{R} = 0.5 (R_{\text{max}} - R_{\text{min}}) \left(\tanh \frac{p - p_{\text{open}}}{\epsilon} + 1 \right) + R_{\text{min}} \quad (70)$$

Remark: Smooth but potentially non-convex flow-pressure relationship!

Valve model `smooth_pres_momentum`:

$$q(p - p_{\text{open}}) = \begin{cases} \frac{p - p_{\text{open}}}{R_{\text{max}}}, & p < p_{\text{open}} - 0.5\epsilon \\ h_{00}p_0 + h_{10}m_0\epsilon + h_{01}p_1 + h_{11}m_1\epsilon, & p \geq p_{\text{open}} - 0.5\epsilon \text{ and } p < p_{\text{open}} + 0.5\epsilon \\ \frac{p - p_{\text{open}}}{R_{\text{min}}}, & p \geq p_{\text{open}} + 0.5\epsilon \end{cases} \quad (71)$$

with

$$p_0 = \frac{-0.5\epsilon}{R_{\text{max}}}, \quad m_0 = \frac{1}{R_{\text{max}}}, \quad p_1 = \frac{0.5\epsilon}{R_{\text{min}}}, \quad m_1 = \frac{1}{R_{\text{min}}} \quad (72)$$

and

$$\begin{aligned} h_{00} &= 2s^3 - 3s^2 + 1, & h_{01} &= -2s^3 + 3s^2, \\ h_{10} &= s^3 - 2s^2 + s, & h_{11} &= s^3 - s^2 \end{aligned} \quad (73)$$

with

$$s = \frac{p - p_{\text{open}} + 0.5\epsilon}{\epsilon} \quad (74)$$

Remarks:

- Collapses to valve model `pmlin_pres` for $\epsilon = 0$
- Smooth and convex flow-pressure relationship

Valve model `pw_pres_regurg`:

$$q(p - p_{\text{open}}) = \begin{cases} cA_o\sqrt{p - p_{\text{open}}}, & p < p_{\text{open}} \\ \frac{p - p_{\text{open}}}{R_{\text{min}}}, & p \geq p_{\text{open}} \end{cases} \quad (75)$$

Remark: Model to allow a regurgitant valve in the closed state, degree of regurgitation can be varied by specifying the valve regurgitant area A_o

- Coronary circulation model: `ZCRp_CRd_lr`

$$\begin{aligned} C_{\text{cor,p}}^{\text{sys},\ell} \left(\frac{dp_{\text{ar}}^{\text{sys},\ell}}{dt} - Z_{\text{cor,p}}^{\text{sys},\ell} \frac{dq_{\text{cor,p,in}}^{\text{sys},\ell}}{dt} \right) &= q_{\text{cor,p,in}}^{\text{sys},\ell} - q_{\text{cor,p}}^{\text{sys},\ell} && \text{left coronary proximal flow balance} \\ R_{\text{cor,p}}^{\text{sys},\ell} q_{\text{cor,p}}^{\text{sys},\ell} &= p_{\text{ar}}^{\text{sys}} - p_{\text{cor,d}}^{\text{sys},\ell} - Z_{\text{cor,p}}^{\text{sys},\ell} q_{\text{cor,p,in}}^{\text{sys},\ell} && \text{left coronary proximal momentum} \\ C_{\text{cor,d}}^{\text{sys},\ell} \frac{d(p_{\text{cor,d}}^{\text{sys},\ell} - p_v^\ell)}{dt} &= q_{\text{cor,p}}^{\text{sys},\ell} - q_{\text{cor,d}}^{\text{sys},\ell} && \text{left coronary distal flow balance} \\ R_{\text{cor,d}}^{\text{sys},\ell} q_{\text{cor,d}}^{\text{sys},\ell} &= p_{\text{cor,d}}^{\text{sys},\ell} - p_{\text{at}}^r && \text{left coronary distal momentum} \\ C_{\text{cor,p}}^{\text{sys},r} \left(\frac{dp_{\text{ar}}^{\text{sys},r}}{dt} - Z_{\text{cor,p}}^{\text{sys},r} \frac{dq_{\text{cor,p,in}}^{\text{sys},r}}{dt} \right) &= q_{\text{cor,p,in}}^{\text{sys},r} - q_{\text{cor,p}}^{\text{sys},r} && \text{right coronary proximal flow balance} \\ R_{\text{cor,p}}^{\text{sys},r} q_{\text{cor,p}}^{\text{sys},r} &= p_{\text{ar}}^{\text{sys}} - p_{\text{cor,d}}^{\text{sys},r} - Z_{\text{cor,p}}^{\text{sys},r} q_{\text{cor,p,in}}^{\text{sys},r} && \text{right coronary proximal momentum} \\ C_{\text{cor,d}}^{\text{sys},r} \frac{d(p_{\text{cor,d}}^{\text{sys},r} - p_v^\ell)}{dt} &= q_{\text{cor,p}}^{\text{sys},r} - q_{\text{cor,d}}^{\text{sys},r} && \text{right coronary distal flow balance} \\ R_{\text{cor,d}}^{\text{sys},r} q_{\text{cor,d}}^{\text{sys},r} &= p_{\text{cor,d}}^{\text{sys},r} - p_{\text{at}}^r && \text{right coronary distal momentum} \\ 0 &= q_{\text{cor,d}}^{\text{sys},\ell} + q_{\text{cor,d}}^{\text{sys},r} - q_{\text{cor,d,out}}^{\text{sys}} && \text{distal coronary junction flow balance} \end{aligned}$$

– Coronary circulation model: ZCRp_CRd

$$\begin{aligned}
C_{\text{cor,p}}^{\text{sys}} \left(\frac{dp_{\text{ar}}^{\text{sys}}}{dt} - Z_{\text{cor,p}}^{\text{sys}} \frac{dq_{\text{cor,p,in}}^{\text{sys}}}{dt} \right) &= q_{\text{cor,p,in}}^{\text{sys}} - q_{\text{cor,p}}^{\text{sys}} && \text{coronary proximal flow balance} \\
R_{\text{cor,p}}^{\text{sys}} q_{\text{cor,p}}^{\text{sys}} &= p_{\text{ar}}^{\text{sys}} - p_{\text{cor,d}}^{\text{sys}} - Z_{\text{cor,p}}^{\text{sys}} q_{\text{cor,p,in}}^{\text{sys}} && \text{coronary proximal momentum} \\
C_{\text{cor,d}}^{\text{sys}} \frac{d(p_{\text{cor,d}}^{\text{sys}} - p_{\text{v}}^{\ell})}{dt} &= q_{\text{cor,p}}^{\text{sys}} - q_{\text{cor,d}}^{\text{sys}} && \text{coronary distal flow balance} \\
R_{\text{cor,d}}^{\text{sys}} q_{\text{cor,d}}^{\text{sys}} &= p_{\text{cor,d}}^{\text{sys}} - p_{\text{at}}^r && \text{coronary distal momentum}
\end{aligned}$$

Systemic and pulmonary circulation, including capillary flow

– Model type : syspulcap, cf. [10], p. 51ff.

$$\begin{aligned}
-Q_{\text{at}}^{\ell} &= q_{\text{ven}}^{\text{pul}} - q_{\text{v,in}}^{\ell} && \text{left atrium flow balance} \\
\tilde{R}_{\text{v,in}}^{\ell} q_{\text{v,in}}^{\ell} &= p_{\text{at}}^{\ell} - p_{\text{v}}^{\ell} && \text{mitral valve momentum} \\
-Q_{\text{v}}^{\ell} &= q_{\text{v,in}}^{\ell} - q_{\text{v,out}}^{\ell} && \text{left ventricle flow balance} \\
\tilde{R}_{\text{v,out}}^{\ell} q_{\text{v,out}}^{\ell} &= p_{\text{v}}^{\ell} - p_{\text{ar}}^{\text{sys}} && \text{aortic valve momentum} \\
0 &= q_{\text{v,out}}^{\ell} - q_{\text{ar,p}}^{\text{sys}} && \text{aortic root flow balance} \\
I_{\text{ar}}^{\text{sys}} \frac{dq_{\text{ar,p}}^{\text{sys}}}{dt} + Z_{\text{ar}}^{\text{sys}} q_{\text{ar,p}}^{\text{sys}} &= p_{\text{ar}}^{\text{sys}} - p_{\text{ar,d}}^{\text{sys}} && \text{aortic root inertia} \\
C_{\text{ar}}^{\text{sys}} \frac{dp_{\text{ar,d}}^{\text{sys}}}{dt} &= q_{\text{ar,p}}^{\text{sys}} - q_{\text{ar}}^{\text{sys}} && \text{systemic arterial flow balance} \\
L_{\text{ar}}^{\text{sys}} \frac{dq_{\text{ar}}^{\text{sys}}}{dt} + R_{\text{ar}}^{\text{sys}} q_{\text{ar}}^{\text{sys}} &= p_{\text{ar,d}}^{\text{sys}} - p_{\text{ar,peri}}^{\text{sys}} && \text{systemic arterial momentum} \\
\left(\sum_{j \in \{ \text{spl,espl, msc,cer,cor} \}} C_{\text{ar},j}^{\text{sys}} \right) \frac{dp_{\text{ar,peri}}^{\text{sys}}}{dt} &= q_{\text{ar}}^{\text{sys}} - \sum_{j \in \{ \text{spl,espl, msc,cer,cor} \}} q_{\text{ar},j}^{\text{sys}} && \text{systemic capillary arterial flow balance} \\
R_{\text{ar},i}^{\text{sys}} q_{\text{ar},i}^{\text{sys}} &= p_{\text{ar,peri}}^{\text{sys}} - p_{\text{ven},i}^{\text{sys}}, \quad i \in \{ \text{spl,espl, msc,cer,cor} \} && \text{systemic capillary arterial momentum} \\
C_{\text{ven},i}^{\text{sys}} \frac{dp_{\text{ven},i}^{\text{sys}}}{dt} &= q_{\text{ar},i}^{\text{sys}} - q_{\text{ven},i}^{\text{sys}}, \quad i \in \{ \text{spl,espl, msc,cer,cor} \} && \text{systemic capillary venous flow balance} \\
R_{\text{ven},i}^{\text{sys}} q_{\text{ven},i}^{\text{sys}} &= p_{\text{ven},i}^{\text{sys}} - p_{\text{ven}}^{\text{sys}}, \quad i \in \{ \text{spl,espl, msc,cer,cor} \} && \text{systemic capillary venous momentum} \\
C_{\text{ven}}^{\text{sys}} \frac{dp_{\text{ven}}^{\text{sys}}}{dt} &= \sum_{j = \text{spl,espl, msc,cer,cor}} q_{\text{ven},j}^{\text{sys}} - q_{\text{ven}}^{\text{sys}} && \text{systemic venous flow balance} \\
L_{\text{ven}}^{\text{sys}} \frac{dq_{\text{ven}}^{\text{sys}}}{dt} + R_{\text{ven}}^{\text{sys}} q_{\text{ven}}^{\text{sys}} &= p_{\text{ven}}^{\text{sys}} - p_{\text{at}}^r && \text{systemic venous momentum}
\end{aligned}$$

$-Q_{\text{at}}^r = q_{\text{ven}}^{\text{sys}} - q_{\text{v},\text{in}}^r$	right atrium flow balance
$\tilde{R}_{\text{v},\text{in}}^r q_{\text{v},\text{in}}^r = p_{\text{at}}^r - p_{\text{v}}^r$	tricuspid valve momentum
$-Q_{\text{v}}^r = q_{\text{v},\text{in}}^r - q_{\text{v},\text{out}}^r$	right ventricle flow balance
$\tilde{R}_{\text{v},\text{out}}^r q_{\text{v},\text{out}}^r = p_{\text{v}}^r - p_{\text{ar}}^{\text{pul}}$	pulmonary valve momentum
$C_{\text{ar}}^{\text{pul}} \frac{dp_{\text{ar}}^{\text{pul}}}{dt} = q_{\text{v},\text{out}}^r - q_{\text{ar}}^{\text{pul}}$	pulmonary arterial flow balance
$L_{\text{ar}}^{\text{pul}} \frac{dq_{\text{ar}}^{\text{pul}}}{dt} + R_{\text{ar}}^{\text{pul}} q_{\text{ar}}^{\text{pul}} = p_{\text{ar}}^{\text{pul}} - p_{\text{cap}}^{\text{pul}}$	pulmonary arterial momentum
$C_{\text{cap}}^{\text{pul}} \frac{dp_{\text{cap}}^{\text{pul}}}{dt} = q_{\text{ar}}^{\text{pul}} - q_{\text{cap}}^{\text{pul}}$	pulmonary capillary flow balance
$R_{\text{cap}}^{\text{pul}} q_{\text{cap}}^{\text{pul}} = p_{\text{cap}}^{\text{pul}} - p_{\text{ven}}^{\text{pul}}$	pulmonary capillary momentum
$C_{\text{ven}}^{\text{pul}} \frac{dp_{\text{ven}}^{\text{pul}}}{dt} = q_{\text{cap}}^{\text{pul}} - q_{\text{ven}}^{\text{pul}}$	pulmonary venous flow balance
$L_{\text{ven}}^{\text{pul}} \frac{dq_{\text{ven}}^{\text{pul}}}{dt} + R_{\text{ven}}^{\text{pul}} q_{\text{ven}}^{\text{pul}} = p_{\text{ven}}^{\text{pul}} - p_{\text{at}}^{\ell}$	pulmonary venous momentum

with:

$$Q_{\text{at}}^{\ell} := -\frac{dV_{\text{at}}^{\ell}}{dt}, \quad Q_{\text{v}}^{\ell} := -\frac{dV_{\text{v}}^{\ell}}{dt}, \quad Q_{\text{at}}^r := -\frac{dV_{\text{at}}^r}{dt}, \quad Q_{\text{v}}^r := -\frac{dV_{\text{v}}^r}{dt}$$

Systemic and pulmonary circulation, including capillary and coronary flow

- Model type : `syspulcapcor`
- Variant of `syspulcap`, with coronaries branching off directly after aortic valve

$-Q_{\text{at}}^\ell = q_{\text{ven}}^{\text{pul}} - q_{\text{v,in}}^\ell$	left atrium flow balance
$\tilde{R}_{\text{v,in}}^\ell q_{\text{v,in}}^\ell = p_{\text{at}}^\ell - p_{\text{v}}^\ell$	mitral valve momentum
$-Q_{\text{v}}^\ell = q_{\text{v,in}}^\ell - q_{\text{v,out}}^\ell$	left ventricle flow balance
$\tilde{R}_{\text{v,out}}^\ell q_{\text{v,out}}^\ell = p_{\text{v}}^\ell - p_{\text{ar}}^{\text{sys}}$	aortic valve momentum
$0 = q_{\text{v,out}}^\ell - q_{\text{ar,p}}^{\text{sys}} - q_{\text{ar,cor,in}}^{\text{sys}}$	aortic root flow balance
$I_{\text{ar}}^{\text{sys}} \frac{dq_{\text{ar,p}}^{\text{sys}}}{dt} + Z_{\text{ar}}^{\text{sys}} q_{\text{ar,p}}^{\text{sys}} = p_{\text{ar}}^{\text{sys}} - p_{\text{ar,d}}^{\text{sys}}$	aortic root inertia
$C_{\text{ar,cor}}^{\text{sys}} \frac{dp_{\text{ar}}^{\text{sys}}}{dt} = q_{\text{ar,cor,in}}^{\text{sys}} - q_{\text{ar,cor}}^{\text{sys}}$	systemic arterial coronary flow balance
$R_{\text{ar,cor}}^{\text{sys}} q_{\text{ar,cor}}^{\text{sys}} = p_{\text{ar}}^{\text{sys}} - p_{\text{ven,cor}}^{\text{sys}}$	systemic arterial coronary momentum
$C_{\text{ar}}^{\text{sys}} \frac{dp_{\text{ar,d}}^{\text{sys}}}{dt} = q_{\text{ar,p}}^{\text{sys}} - q_{\text{ar}}^{\text{sys}}$	systemic arterial flow balance
$L_{\text{ar}}^{\text{sys}} \frac{dq_{\text{ar}}^{\text{sys}}}{dt} + R_{\text{ar}}^{\text{sys}} q_{\text{ar}}^{\text{sys}} = p_{\text{ar,d}}^{\text{sys}} - p_{\text{ar,peri}}^{\text{sys}}$	systemic arterial flow balance
$\left(\sum_{j \in \left\{ \begin{smallmatrix} \text{spl,espl,} \\ \text{msc,cer} \end{smallmatrix} \right\}} C_{\text{ar},j}^{\text{sys}} \right) \frac{dp_{\text{ar,peri}}^{\text{sys}}}{dt} = q_{\text{ar}}^{\text{sys}} - \sum_{j \in \left\{ \begin{smallmatrix} \text{spl,espl,} \\ \text{msc,cer} \end{smallmatrix} \right\}} q_{\text{ar},j}^{\text{sys}}$	systemic arterial capillary flow balance
$R_{\text{ar},i}^{\text{sys}} q_{\text{ar},i}^{\text{sys}} = p_{\text{ar,peri}}^{\text{sys}} - p_{\text{ven},i}^{\text{sys}}, \quad i \in \left\{ \begin{smallmatrix} \text{spl,espl,} \\ \text{msc,cer} \end{smallmatrix} \right\}$	systemic arterial capillary momentum
$C_{\text{ven},i}^{\text{sys}} \frac{dp_{\text{ven},i}^{\text{sys}}}{dt} = q_{\text{ar},i}^{\text{sys}} - q_{\text{ven},i}^{\text{sys}}, \quad i \in \left\{ \begin{smallmatrix} \text{spl,espl,} \\ \text{msc,cer} \end{smallmatrix} \right\}$	systemic venous capillary flow balance
$R_{\text{ven},i}^{\text{sys}} q_{\text{ven},i}^{\text{sys}} = p_{\text{ven},i}^{\text{sys}} - p_{\text{ven}}^{\text{sys}}, \quad i \in \left\{ \begin{smallmatrix} \text{spl,espl,} \\ \text{msc,cer} \end{smallmatrix} \right\}$	systemic venous capillary momentum
$C_{\text{ven}}^{\text{sys}} \frac{dp_{\text{ven}}^{\text{sys}}}{dt} = \sum_{j = \begin{smallmatrix} \text{spl,espl,} \\ \text{msc,cer} \end{smallmatrix}} q_{\text{ven},j}^{\text{sys}} - q_{\text{ven}}^{\text{sys}}$	systemic venous flow balance
$L_{\text{ven}}^{\text{sys}} \frac{dq_{\text{ven}}^{\text{sys}}}{dt} + R_{\text{ven}}^{\text{sys}} q_{\text{ven}}^{\text{sys}} = p_{\text{ven}}^{\text{sys}} - p_{\text{at}}^r$	systemic venous momentum
$C_{\text{ven,cor}}^{\text{sys}} \frac{dp_{\text{ven,cor}}^{\text{sys}}}{dt} = q_{\text{ar,cor}}^{\text{sys}} - q_{\text{ven,cor}}^{\text{sys}}$	systemic venous coronary flow balance
$R_{\text{ven,cor}}^{\text{sys}} q_{\text{ven,cor}}^{\text{sys}} = p_{\text{ven,cor}}^{\text{sys}} - p_{\text{at}}^r$	systemic venous coronary momentum

$$\begin{aligned}
-Q_{\text{at}}^r &= q_{\text{ven}}^{\text{sys}} + q_{\text{ven,cor}}^{\text{sys}} - q_{\text{v,in}}^r && \text{right atrium flow balance} \\
\tilde{R}_{\text{v,in}}^r q_{\text{v,in}}^r &= p_{\text{at}}^r - p_{\text{v}}^r && \text{tricuspid valve momentum} \\
-Q_{\text{v}}^r &= q_{\text{v,in}}^r - q_{\text{v,out}}^r && \text{right ventricle flow balance} \\
\tilde{R}_{\text{v,out}}^r q_{\text{v,out}}^r &= p_{\text{v}}^r - p_{\text{ar}}^{\text{pul}} && \text{pulmonary valve momentum} \\
C_{\text{ar}}^{\text{pul}} \frac{dp_{\text{ar}}^{\text{pul}}}{dt} &= q_{\text{v,out}}^r - q_{\text{ar}}^{\text{pul}} && \text{pulmonary arterial flow balance} \\
L_{\text{ar}}^{\text{pul}} \frac{dq_{\text{ar}}^{\text{pul}}}{dt} + R_{\text{ar}}^{\text{pul}} q_{\text{ar}}^{\text{pul}} &= p_{\text{ar}}^{\text{pul}} - p_{\text{cap}}^{\text{pul}} && \text{pulmonary arterial momentum} \\
C_{\text{cap}}^{\text{pul}} \frac{dp_{\text{cap}}^{\text{pul}}}{dt} &= q_{\text{ar}}^{\text{pul}} - q_{\text{cap}}^{\text{pul}} && \text{pulmonary capillary flow balance} \\
R_{\text{cap}}^{\text{pul}} q_{\text{cap}}^{\text{pul}} &= p_{\text{cap}}^{\text{pul}} - p_{\text{ven}}^{\text{pul}} && \text{pulmonary capillary momentum} \\
C_{\text{ven}}^{\text{pul}} \frac{dp_{\text{ven}}^{\text{pul}}}{dt} &= q_{\text{cap}}^{\text{pul}} - q_{\text{ven}}^{\text{pul}} && \text{pulmonary venous flow balance} \\
L_{\text{ven}}^{\text{pul}} \frac{dq_{\text{ven}}^{\text{pul}}}{dt} + R_{\text{ven}}^{\text{pul}} q_{\text{ven}}^{\text{pul}} &= p_{\text{ven}}^{\text{pul}} - p_{\text{at}}^{\ell} && \text{pulmonary venous momentum}
\end{aligned}$$

with:

$$Q_{\text{at}}^{\ell} := -\frac{dV_{\text{at}}^{\ell}}{dt}, \quad Q_{\text{v}}^{\ell} := -\frac{dV_{\text{v}}^{\ell}}{dt}, \quad Q_{\text{at}}^r := -\frac{dV_{\text{at}}^r}{dt}, \quad Q_{\text{v}}^r := -\frac{dV_{\text{v}}^r}{dt} \quad (76)$$

Systemic and pulmonary circulation, capillary flow, and respiratory (gas transport + dissociation) model

- Model type : `syspulcaprespir`
- Model equations described in [10], p. 51ff., 58ff.

4.4 Multi-physics coupling

Solid + 0D flow

- Example: Sec. 4.4 and `demos/solid_flow0d`
- Problem type: `solid_flow0d`
- solid momentum in (3) or in (12) augmented by following term:

$$r_{\delta u} \leftarrow r_{\delta u} + \sum_{i=1}^{n_{0d}^b} \Lambda_i \int_{\Gamma_{0,i}^{\text{rs-0d}}} J \mathbf{F}^{-\text{T}} \mathbf{n}_0 \cdot \delta \mathbf{u} \, dA \quad (77)$$

– Multiplier constraint

$$r_{\delta\Lambda} \left(\{\Lambda\}_{n_{0d}^b}, \mathbf{u}; \{\delta\Lambda\}_{n_{0d}^b} \right) := \sum_{i=1}^{n_{0d}^b} \left(\int_{\Gamma_{0,i}^{s-0d}} \mathbf{v}(\mathbf{u}) \cdot J\mathbf{F}^{-T} \mathbf{n}_0 \, dA - \alpha_i \mathbf{y} \left(\{\Lambda\}_{n_{0d}^b} \right) \cdot \mathbf{e}_i \right) \delta\Lambda_i \quad \forall \delta\Lambda_i \quad (78)$$

– Discrete nonlinear system to solve in each time step n for displacement-based solid:

$$\mathbf{r}_{n+1} = \begin{bmatrix} \mathbf{r}_u(\mathbf{u}, \Lambda) \\ \mathbf{r}_\Lambda(\Lambda, \mathbf{u}) \end{bmatrix}_{n+1} = \mathbf{0} \quad (79)$$

– Discrete linear system to solve in each Newton iteration k for displacement-based solid:

$$\begin{bmatrix} \mathbf{K}_{uu} & \mathbf{K}_{u\Lambda} \\ \mathbf{K}_{\Lambda u} & \mathbf{K}_{\Lambda\Lambda} \end{bmatrix}_{n+1}^k \begin{bmatrix} \Delta \mathbf{u} \\ \Delta \Lambda \end{bmatrix}_{n+1}^{k+1} = - \begin{bmatrix} \mathbf{r}_u \\ \mathbf{r}_\Lambda \end{bmatrix}_{n+1}^k \quad (80)$$

– Discrete nonlinear system to solve in each time step n for incompressible solid:

$$\mathbf{r}_{n+1} = \begin{bmatrix} \mathbf{r}_u(\mathbf{u}, \mathbf{p}, \Lambda) \\ \mathbf{r}_p(\mathbf{u}) \\ \mathbf{r}_\Lambda(\Lambda, \mathbf{u}) \end{bmatrix}_{n+1} = \mathbf{0} \quad (81)$$

– Discrete linear system to solve in each Newton iteration k for incompressible solid:

$$\begin{bmatrix} \mathbf{K}_{uu} & \mathbf{K}_{up} & \mathbf{K}_{u\Lambda} \\ \mathbf{K}_{pu} & \mathbf{0} & \mathbf{0} \\ \mathbf{K}_{\Lambda u} & \mathbf{0} & \mathbf{K}_{\Lambda\Lambda} \end{bmatrix}_{n+1}^k \begin{bmatrix} \Delta \mathbf{u} \\ \Delta \mathbf{p} \\ \Delta \Lambda \end{bmatrix}_{n+1}^{k+1} = - \begin{bmatrix} \mathbf{r}_u \\ \mathbf{r}_p \\ \mathbf{r}_\Lambda \end{bmatrix}_{n+1}^k \quad (82)$$

– sub-solves: 0D model has to hold true in each nonlinear iteration k :

$$\mathbf{r}^{0D}(\Lambda)|_k = \mathbf{0}, \quad \rightsquigarrow \mathbf{K}_k^{0D,j} \Delta \mathbf{q}_k^{j+1} = -\mathbf{r}_k^{0D,j} \quad (83)$$

Fluid + 0D flow

– Example: Sec. 4.4 demos/fluid_flow0d

– Problem type: fluid_flow0d

– fluid momentum in (35) augmented by following term:

$$r_{\delta v} \leftarrow r_{\delta v} + \sum_{i=1}^{n_{0d}^b} \Lambda_i \int_{\Gamma_i^{f-0d}} \mathbf{n} \cdot \delta \mathbf{v} \, dA \quad (84)$$

– Multiplier constraint

$$r_{\delta\Lambda} \left(\{\Lambda\}_{n_{0d}^b}, \mathbf{v}; \{\delta\Lambda\}_{n_{0d}^b} \right) := \sum_{i=1}^{n_{0d}^b} \left(\int_{\Gamma_i^{f-0d}} \mathbf{v} \cdot \mathbf{n} \, dA - \alpha_i \mathbf{y} \left(\{\Lambda\}_{n_{0d}^b} \right) \cdot \mathbf{e}_i \right) \delta\Lambda_i \quad \forall \delta\Lambda_i \quad (85)$$

– Discrete nonlinear system to solve in each time step n :

$$\mathbf{r}_{n+1} = \begin{bmatrix} \mathbf{r}_v(\mathbf{v}, \mathbf{p}, \boldsymbol{\Lambda}) \\ \mathbf{r}_p(\mathbf{p}, \mathbf{v}) \\ \mathbf{r}_\Lambda(\boldsymbol{\Lambda}, \mathbf{v}) \end{bmatrix}_{n+1} = \mathbf{0} \quad (86)$$

– Discrete linear system to solve in each Newton iteration k :

$$\begin{bmatrix} \mathbf{K}_{vv} & \mathbf{K}_{vp} & \mathbf{K}_{v\Lambda} \\ \mathbf{K}_{pv} & \mathbf{K}_{pp} & \mathbf{0} \\ \mathbf{K}_{\Lambda v} & \mathbf{0} & \mathbf{K}_{\Lambda\Lambda} \end{bmatrix}_{n+1}^k \begin{bmatrix} \Delta \mathbf{v} \\ \Delta \mathbf{p} \\ \Delta \boldsymbol{\Lambda} \end{bmatrix}_{n+1}^{k+1} = - \begin{bmatrix} \mathbf{r}_v \\ \mathbf{r}_p \\ \mathbf{r}_\Lambda \end{bmatrix}_{n+1}^k \quad (87)$$

ALE fluid + 0D flow

– Problem type: `fluid_ale_flow0d`

– fluid momentum in (53) augmented by following term:

$$r_{\delta v} \leftarrow r_{\delta v} + \sum_{i=1}^{n_{0d}^b} \Lambda_i \int_{\Gamma_{0,i}^{f-0d}} \widehat{\mathbf{J}} \widehat{\mathbf{F}}^{-T} \mathbf{n}_0 \cdot \delta \mathbf{v} \, dA \quad (88)$$

– Multiplier constraint

$$r_{\delta\Lambda} \left(\{\Lambda\}_{n_{0d}^b}, \mathbf{v}, \mathbf{d}; \{\delta\Lambda\}_{n_{0d}^b} \right) := \sum_{i=1}^{n_{0d}^b} \left(\int_{\Gamma_i^{f-0d}} (\mathbf{v} - \widehat{\mathbf{w}}) \cdot \widehat{\mathbf{J}} \widehat{\mathbf{F}}^{-T} \mathbf{n}_0 \, dA - \alpha_i \mathbf{y} \left(\{\Lambda\}_{n_{0d}^b} \right) \cdot \mathbf{e}_i \right) \delta\Lambda_i \quad \forall \delta\Lambda_i \quad (89)$$

with $\mathbf{w}(\mathbf{d}) = \frac{\partial \mathbf{d}}{\partial t}$

– Discrete nonlinear system to solve in each time step n :

$$\mathbf{r}_{n+1} = \begin{bmatrix} \mathbf{r}_v(\mathbf{v}, \mathbf{p}, \boldsymbol{\Lambda}, \mathbf{d}) \\ \mathbf{r}_p(\mathbf{p}, \mathbf{v}, \mathbf{d}) \\ \mathbf{r}_\Lambda(\boldsymbol{\Lambda}, \mathbf{v}, \mathbf{d}) \\ \mathbf{r}_d(\mathbf{d}) \end{bmatrix}_{n+1} = \mathbf{0} \quad (90)$$

- Discrete linear system to solve in each Newton iteration k :

$$\begin{bmatrix} \mathbf{K}_{vv} & \mathbf{K}_{vp} & \mathbf{K}_{v\Lambda} & \mathbf{K}_{vd} \\ \mathbf{K}_{pv} & \mathbf{K}_{pp} & \mathbf{0} & \mathbf{K}_{pd} \\ \mathbf{K}_{\Lambda v} & \mathbf{0} & \mathbf{K}_{\Lambda\Lambda} & \mathbf{K}_{\Lambda d} \\ \mathbf{K}_{dv} & \mathbf{0} & \mathbf{0} & \mathbf{K}_{dd} \end{bmatrix}_{n+1}^k \begin{bmatrix} \Delta \mathbf{v} \\ \Delta \mathbf{p} \\ \Delta \boldsymbol{\Lambda} \\ \Delta \mathbf{d} \end{bmatrix}_{n+1}^{k+1} = - \begin{bmatrix} \mathbf{r}_v \\ \mathbf{r}_p \\ \mathbf{r}_\Lambda \\ \mathbf{r}_d \end{bmatrix}_{n+1}^k \quad (91)$$

Fluid-Solid Interaction (FSI)

- Example: Sec. 5.6 and `demofsi`
- Problem type: `fsi`
- Solid momentum in (3) or (12) augmented by following term:

$$r_u \leftarrow r_u + \int_{\Gamma_0^{\text{f-s}}} \boldsymbol{\lambda} \cdot \delta \mathbf{u} \, dA \quad (92)$$

- Fluid momentum in (53) is augmented by following term:

$$r_v \leftarrow r_v - \int_{\Gamma_0^{\text{f-s}}} \boldsymbol{\lambda} \cdot \delta \mathbf{v} \, dA \quad (93)$$

Note the different signs (actio=reactio!)

- Lagrange multiplier constraint:

- “solid-governed”:

$$r_\lambda(\mathbf{u}, \mathbf{v}; \delta \boldsymbol{\lambda}) := \int_{\Gamma_0^{\text{f-s}}} (\mathbf{u} - \mathbf{u}_f(\mathbf{v})) \cdot \delta \boldsymbol{\lambda} \, dA, \quad \forall \delta \boldsymbol{\lambda} \quad (94)$$

- “fluid-governed”:

$$r_\lambda(\mathbf{v}, \mathbf{u}; \delta \boldsymbol{\lambda}) := \int_{\Gamma_0^{\text{f-s}}} \left(\mathbf{v} - \frac{d\mathbf{u}}{dt} \right) \cdot \delta \boldsymbol{\lambda} \, dA, \quad \forall \delta \boldsymbol{\lambda} \quad (95)$$

- Discrete nonlinear system to solve in each time step n for displacement-based solid:

$$\mathbf{r}_{n+1} = \begin{bmatrix} \mathbf{r}_u^s(\mathbf{u}, \boldsymbol{\lambda}) \\ \mathbf{r}_v^f(\mathbf{v}, \mathbf{p}, \boldsymbol{\lambda}, \mathbf{d}) \\ \mathbf{r}_p^f(\mathbf{p}, \mathbf{v}, \mathbf{d}) \\ \mathbf{r}_\lambda(\mathbf{u}, \mathbf{v}) \\ \mathbf{r}_d(\mathbf{d}) \end{bmatrix}_{n+1} = \mathbf{0} \quad (96)$$

- Discrete nonlinear system to solve in each time step n for incompressible solid:

$$\mathbf{r}_{n+1} = \begin{bmatrix} \mathbf{r}_u^s(\mathbf{u}, \mathbf{p}^s, \boldsymbol{\lambda}) \\ \mathbf{r}_p^s(\mathbf{u}) \\ \mathbf{r}_v^f(\mathbf{v}, \mathbf{p}, \boldsymbol{\lambda}, \mathbf{d}) \\ \mathbf{r}_p^f(\mathbf{p}, \mathbf{v}, \mathbf{d}) \\ \mathbf{r}_\lambda(\mathbf{u}, \mathbf{v}) \\ \mathbf{r}_d(\mathbf{d}) \end{bmatrix}_{n+1} = \mathbf{0} \quad (97)$$

- Discrete linear system to solve in each Newton iteration k for displacement-based solid:

$$\begin{bmatrix} \mathbf{K}_{uu} & \mathbf{0} & \mathbf{0} & \mathbf{K}_{u\lambda} & \mathbf{0} \\ \mathbf{0} & \mathbf{K}_{vv} & \mathbf{K}_{vp} & \mathbf{K}_{v\lambda} & \mathbf{K}_{vd} \\ \mathbf{0} & \mathbf{K}_{pv} & \mathbf{K}_{pp} & \mathbf{0} & \mathbf{K}_{pd} \\ \mathbf{K}_{\lambda u} & \mathbf{K}_{\lambda v} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{K}_{dv} & \mathbf{0} & \mathbf{0} & \mathbf{K}_{dd} \end{bmatrix}_{n+1}^k \begin{bmatrix} \Delta \mathbf{u} \\ \Delta \mathbf{v} \\ \Delta \mathbf{p} \\ \Delta \boldsymbol{\lambda} \\ \Delta \mathbf{d} \end{bmatrix}_{n+1}^{k+1} = - \begin{bmatrix} \mathbf{r}_u^s \\ \mathbf{r}_v^f \\ \mathbf{r}_p^f \\ \mathbf{r}_\lambda \\ \mathbf{r}_d \end{bmatrix}_{n+1}^k \quad (98)$$

- Discrete linear system to solve in each Newton iteration k for incompressible solid:

$$\begin{bmatrix} \mathbf{K}_{uu} & \mathbf{K}_{up} & \mathbf{0} & \mathbf{0} & \mathbf{K}_{u\lambda} & \mathbf{0} \\ \mathbf{K}_{pu} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{K}_{vv} & \mathbf{K}_{vp} & \mathbf{K}_{v\lambda} & \mathbf{K}_{vd} \\ \mathbf{0} & \mathbf{0} & \mathbf{K}_{pv} & \mathbf{K}_{pp} & \mathbf{0} & \mathbf{K}_{pd} \\ \mathbf{K}_{\lambda u} & \mathbf{0} & \mathbf{K}_{\lambda v} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{K}_{dv} & \mathbf{0} & \mathbf{0} & \mathbf{K}_{dd} \end{bmatrix}_{n+1}^k \begin{bmatrix} \Delta \mathbf{u} \\ \Delta \mathbf{p} \\ \Delta \mathbf{v} \\ \Delta \mathbf{p} \\ \Delta \boldsymbol{\lambda} \\ \Delta \mathbf{d} \end{bmatrix}_{n+1}^{k+1} = - \begin{bmatrix} \mathbf{r}_u^s \\ \mathbf{r}_p^s \\ \mathbf{r}_v^f \\ \mathbf{r}_p^f \\ \mathbf{r}_\lambda \\ \mathbf{r}_d \end{bmatrix}_{n+1}^k \quad (99)$$

Fluid-Solid Interaction (FSI) + 0D flow

– Problem type: `fsi_flow0d`

– Discrete nonlinear system to solve in each time step n for displacement-based solid:

$$\mathbf{r}_{n+1} = \begin{bmatrix} \mathbf{r}_u^s(\mathbf{u}, \boldsymbol{\lambda}) \\ \mathbf{r}_v^f(\mathbf{v}, \mathbf{p}, \boldsymbol{\lambda}, \boldsymbol{\Lambda}, \mathbf{d}) \\ \mathbf{r}_p^f(\mathbf{p}, \mathbf{v}, \mathbf{d}) \\ \mathbf{r}_\lambda(\mathbf{u}, \mathbf{v}) \\ \mathbf{r}_\Lambda(\boldsymbol{\Lambda}, \mathbf{v}, \mathbf{d}) \\ \mathbf{r}_d(\mathbf{d}) \end{bmatrix}_{n+1} = \mathbf{0} \quad (100)$$

– Discrete nonlinear system to solve in each time step n for incompressible solid:

$$\mathbf{r}_{n+1} = \begin{bmatrix} \mathbf{r}_u^s(\mathbf{u}, \mathbf{p}^s, \boldsymbol{\lambda}) \\ \mathbf{r}_p^s(\mathbf{u}) \\ \mathbf{r}_v^f(\mathbf{v}, \mathbf{p}, \boldsymbol{\lambda}, \boldsymbol{\Lambda}, \mathbf{d}) \\ \mathbf{r}_p^f(\mathbf{p}, \mathbf{v}, \mathbf{d}) \\ \mathbf{r}_\lambda(\mathbf{u}, \mathbf{v}) \\ \mathbf{r}_\Lambda(\boldsymbol{\Lambda}, \mathbf{v}, \mathbf{d}) \\ \mathbf{r}_d(\mathbf{d}) \end{bmatrix}_{n+1} = \mathbf{0} \quad (101)$$

– Discrete linear system to solve in each Newton iteration k for displacement-based solid:

$$\begin{bmatrix} \mathbf{K}_{uu} & \mathbf{0} & \mathbf{0} & \mathbf{K}_{u\lambda} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{K}_{vv} & \mathbf{K}_{vp} & \mathbf{K}_{v\lambda} & \mathbf{K}_{v\Lambda} & \mathbf{K}_{vd} \\ \mathbf{0} & \mathbf{K}_{pv} & \mathbf{K}_{pp} & \mathbf{0} & \mathbf{0} & \mathbf{K}_{pd} \\ \mathbf{K}_{\lambda u} & \mathbf{K}_{\lambda v} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{K}_{\Lambda v} & \mathbf{0} & \mathbf{0} & \mathbf{K}_{\Lambda\Lambda} & \mathbf{K}_{\Lambda d} \\ \mathbf{0} & \mathbf{K}_{dv} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{K}_{dd} \end{bmatrix}_{n+1}^k \begin{bmatrix} \Delta \mathbf{u} \\ \Delta \mathbf{v} \\ \Delta \mathbf{p} \\ \Delta \boldsymbol{\lambda} \\ \Delta \boldsymbol{\Lambda} \\ \Delta \mathbf{d} \end{bmatrix}_{n+1}^{k+1} = - \begin{bmatrix} \mathbf{r}_u^s \\ \mathbf{r}_v^f \\ \mathbf{r}_p^f \\ \mathbf{r}_\lambda \\ \mathbf{r}_\Lambda \\ \mathbf{r}_d \end{bmatrix}_{n+1}^k \quad (102)$$

– Discrete linear system to solve in each Newton iteration k for incompressible solid:

$$\begin{bmatrix} \mathbf{K}_{uu} & \mathbf{K}_{up} & \mathbf{0} & \mathbf{0} & \mathbf{K}_{u\lambda} & \mathbf{0} & \mathbf{0} \\ \mathbf{K}_{pu} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{K}_{vv} & \mathbf{K}_{vp} & \mathbf{K}_{v\lambda} & \mathbf{K}_{v\Lambda} & \mathbf{K}_{vd} \\ \mathbf{0} & \mathbf{0} & \mathbf{K}_{pv} & \mathbf{K}_{pp} & \mathbf{0} & \mathbf{0} & \mathbf{K}_{pd} \\ \mathbf{K}_{\lambda u} & \mathbf{0} & \mathbf{K}_{\lambda v} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{0} \\ \mathbf{0} & \mathbf{0} & \mathbf{K}_{\Lambda v} & \mathbf{0} & \mathbf{0} & \mathbf{K}_{\Lambda\Lambda} & \mathbf{K}_{\Lambda d} \\ \mathbf{0} & \mathbf{0} & \mathbf{K}_{dv} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \mathbf{K}_{dd} \end{bmatrix}_{n+1}^k \begin{bmatrix} \Delta \mathbf{u} \\ \Delta \mathbf{p} \\ \Delta \mathbf{v} \\ \Delta \mathbf{p} \\ \Delta \lambda \\ \Delta \Lambda \\ \Delta \mathbf{d} \end{bmatrix}_{n+1}^{k+1} = - \begin{bmatrix} \mathbf{r}_u^s \\ \mathbf{r}_p^s \\ \mathbf{r}_v^f \\ \mathbf{r}_p^f \\ \mathbf{r}_\lambda \\ \mathbf{r}_\Lambda \\ \mathbf{r}_d \end{bmatrix}_{n+1}^k \quad (103)$$

Fluid-reduced-Solid Interaction (FrSI)

– Boundary term:

$$r_{\tilde{s}} := \int_{\Gamma_0^{\text{f-s}}} h_0 \left[\rho_{0,s} \frac{\partial \mathbf{v}}{\partial t} \cdot \delta \mathbf{v} + \tilde{\mathbf{P}}(\mathbf{u}_{\text{f}}(\mathbf{v}) + \tilde{\mathbf{u}}_{\text{pre}}, \mathbf{v}) : \tilde{\nabla}_0 \delta \mathbf{v} \right] dA \quad (104)$$

– Discrete nonlinear system to solve in each time step n :

$$\mathbf{r}_{n+1} = \begin{bmatrix} \mathbf{V}_v^{\Gamma^T} \mathbf{r}_v(\mathbf{V}_v^{\Gamma} \tilde{\mathbf{v}}, \mathbf{p}, \Lambda, \mathbf{d}) \\ \mathbf{r}_p(\mathbf{p}, \mathbf{V}_v^{\Gamma} \tilde{\mathbf{v}}, \mathbf{d}) \\ \mathbf{r}_\Lambda(\Lambda, \mathbf{V}_v^{\Gamma} \tilde{\mathbf{v}}, \mathbf{d}) \\ \mathbf{r}_d(\mathbf{d}, \mathbf{V}_v^{\Gamma} \tilde{\mathbf{v}}) \end{bmatrix}_{n+1} = \mathbf{0} \quad (105)$$

– Discrete linear system to solve in each Newton iteration k :

$$\begin{bmatrix} \mathbf{V}_v^{\Gamma^T} \mathbf{K}_{vv} \mathbf{V}_v^{\Gamma} & \mathbf{V}_v^{\Gamma^T} \mathbf{K}_{vp} & \mathbf{V}_v^{\Gamma^T} \mathbf{K}_{v\Lambda} & \mathbf{V}_v^{\Gamma^T} \mathbf{K}_{vd} \\ \mathbf{K}_{pv} \mathbf{V}_v^{\Gamma} & \mathbf{K}_{pp} & \mathbf{0} & \mathbf{K}_{pd} \\ \mathbf{K}_{\Lambda v} \mathbf{V}_v^{\Gamma} & \mathbf{0} & \mathbf{K}_{\Lambda\Lambda} & \mathbf{K}_{\Lambda d} \\ \mathbf{K}_{dv} \mathbf{V}_v^{\Gamma} & \mathbf{0} & \mathbf{0} & \mathbf{K}_{dd} \end{bmatrix}_{n+1}^k \begin{bmatrix} \Delta \tilde{\mathbf{v}} \\ \Delta \mathbf{p} \\ \Delta \Lambda \\ \Delta \mathbf{d} \end{bmatrix}_{n+1}^{k+1} = - \begin{bmatrix} \mathbf{V}_v^{\Gamma^T} \mathbf{r}_v \\ \mathbf{r}_p \\ \mathbf{r}_\Lambda \\ \mathbf{r}_d \end{bmatrix}_{n+1}^k \quad (106)$$

Multiphase Fluid

- Primary variables: velocity $\mathbf{v}$, pressure p , phase ϕ , and (chem.) potential μ
- conservative form of Navier-Stokes, with capillary force term:

$$\begin{aligned}
\frac{\partial(\rho\mathbf{v})}{\partial t} + \nabla \cdot (\rho(\mathbf{v} \otimes \mathbf{v})) &= \nabla \cdot \boldsymbol{\sigma} - \phi \nabla \mu + \hat{\mathbf{b}} & \text{in } \Omega_t \times [0, T], \\
\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{v}) &= 0 & \text{in } \Omega_t \times [0, T], \\
\mathbf{v} &= \hat{\mathbf{v}} & \text{on } \Gamma_t^D \times [0, T], \\
\mathbf{t} = \boldsymbol{\sigma} \mathbf{n} &= \hat{\mathbf{t}} & \text{on } \Gamma_t^N \times [0, T], \\
\mathbf{v}(\mathbf{x}, 0) &= \hat{\mathbf{v}}_0(\mathbf{x}) & \text{in } \Omega_t,
\end{aligned} \tag{107}$$

with the constitutive equation for the Cauchy stress tensor given by

$$\boldsymbol{\sigma} = -p\mathbf{I} + \eta (\nabla \mathbf{v} + (\nabla \mathbf{v})^T) + \left(\zeta - \frac{2}{d}\eta \right) \nabla \cdot \mathbf{v} \tag{108}$$

- conservative form of the Cahn-Hilliard equations:

$$\begin{aligned}
\frac{\partial \phi}{\partial t} + \nabla \cdot (\phi \mathbf{v}) &= -\nabla \cdot \mathbf{J} & \text{in } \Omega_t \times [0, T], \\
\mu - \frac{d\psi}{d\phi} + \nabla \cdot (\kappa \nabla \phi) &= 0 & \text{in } \Omega_t \times [0, T], \\
\phi(\mathbf{x}, 0) &= \hat{\phi}_0(\mathbf{x}) & \text{in } \Omega_t,
\end{aligned} \tag{109}$$

with the constitutive equation for the diffusive flux

$$\mathbf{J} = -M(\phi) \nabla (\mu + \alpha p) \tag{110}$$

where $\alpha = \frac{\rho_1 - \rho_2}{\rho_1 + \rho_2}$

We assume $\phi \in [a, b]$ and define the normalized phase field variable

$$\chi(\phi) = \frac{\phi - a}{b - a} \tag{111}$$

and fluid density as well as dynamic and bulk viscosity read

$$\rho(\chi(\phi)) = \rho_1(1 - \chi) + \rho_2\chi, \tag{112}$$

$$\eta(\chi(\phi)) = \eta_1(1 - \chi) + \eta_2\chi, \tag{113}$$

$$\zeta(\chi(\phi)) = \zeta_1(1 - \chi) + \zeta_2\chi \tag{114}$$

Chemical potential:

$$\psi(\phi) = D_0(a - \phi)^2(b - \phi)^2 \tag{115}$$

with D_0 a bulk free-energy parameter

Mobility:

$$M(\phi) = M_0 |(a - \phi)^\gamma (b - \phi)^\gamma| \quad (116)$$

Variational form:

Find $(\mathbf{v}, p, \phi, \mu) \in \mathcal{V}_v^{h,D} \times \mathcal{V}_p^{h,D} \times \mathcal{V}_\phi^{h,D} \times \mathcal{V}_\mu^{h,D}$ such that

$$\begin{aligned} \int_{\Omega} \left(\frac{\partial(\rho \mathbf{v})}{\partial t} + \nabla \cdot (\rho(\mathbf{v} \otimes \mathbf{v})) \right) \cdot \delta \mathbf{v} \, dv + \int_{\Omega} \boldsymbol{\sigma} : \nabla \delta \mathbf{v} \, dv + \int_{\Omega} \phi \nabla \mu \cdot \delta \mathbf{v} \, dv - \delta \mathcal{P}_{\text{ext}}(\mathbf{v}; \delta \mathbf{v}) &= 0, \\ \int_{\Omega} \left(\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \mathbf{v}) \right) \delta p \, dv &= 0, \\ \int_{\Omega} \left(\frac{\partial \phi}{\partial t} + \nabla \cdot (\phi \mathbf{v}) \right) \delta \phi \, dv - \int_{\Omega} \mathbf{J} \cdot \nabla \delta \phi \, dv + (\text{Neumann BCs}) &= 0, \\ \int_{\Omega} \mu \delta \mu \, dv - \int_{\Omega} \frac{d\psi}{d\phi} \delta \mu \, dv - \int_{\Omega} \kappa \nabla \phi \cdot \nabla \delta \mu \, dv + (\text{Neumann BCs}) &= 0, \end{aligned} \quad (117)$$

for all $(\delta \mathbf{v}, \delta p, \delta \phi, \delta \mu) \in \mathcal{V}_v^h \times \mathcal{V}_p^h \times \mathcal{V}_\phi^h \times \mathcal{V}_\mu^h$, with

$$\frac{\partial \rho}{\partial t} = \rho'(\phi) \frac{\partial \phi}{\partial t} \quad (118)$$

– Discrete nonlinear system to solve in each time step n :

$$\mathbf{r}_{n+1} = \begin{bmatrix} \mathbf{r}_v(\mathbf{v}, \mathbf{p}, \phi, \mu) \\ \mathbf{r}_p(\mathbf{p}, \mathbf{v}, \phi) \\ \mathbf{r}_\phi(\phi, \mathbf{v}, \mathbf{p}, \mu) \\ \mathbf{r}_\mu(\mu, \phi) \end{bmatrix}_{n+1} = \mathbf{0} \quad (119)$$

– Discrete linear system to solve in each Newton iteration k :

$$\begin{bmatrix} \mathbf{K}_{vv} & \mathbf{K}_{vp} & \mathbf{K}_{v\phi} & \mathbf{K}_{v\mu} \\ \mathbf{K}_{pv} & \mathbf{K}_{pp} & \mathbf{K}_{p\phi} & \mathbf{0} \\ \mathbf{K}_{\phi v} & \mathbf{K}_{\phi p} & \mathbf{K}_{\phi\phi} & \mathbf{K}_{\phi\mu} \\ \mathbf{0} & \mathbf{0} & \mathbf{K}_{\mu\phi} & \mathbf{K}_{\mu\mu} \end{bmatrix}_{n+1}^k \begin{bmatrix} \Delta \mathbf{v} \\ \Delta \mathbf{p} \\ \Delta \phi \\ \Delta \mu \end{bmatrix}_{n+1}^{k+1} = - \begin{bmatrix} \mathbf{r}_v \\ \mathbf{r}_p \\ \mathbf{r}_\phi \\ \mathbf{r}_\mu \end{bmatrix}_{n+1}^k \quad (120)$$

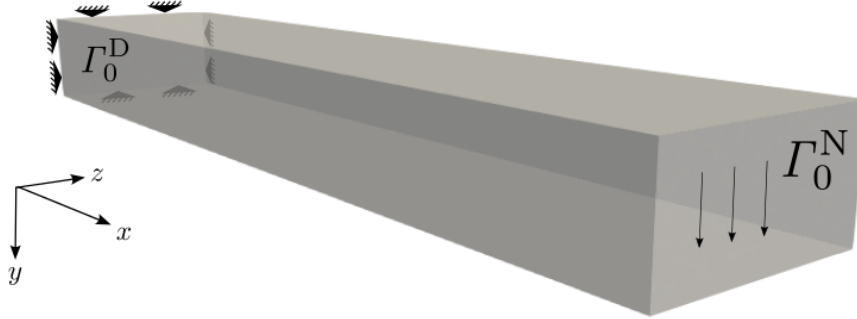


Figure 1: Cantilever, problem setup.

5 Demos

5.1 Demo: Solid

- Physics description given in sec. 4.1
- Input files: `demos/solid/`

Cantilever under tip load

This example demonstrates how to set up a quasi-static solid mechanics elasticity problem. The deformation of a steel cantilever under transverse conservative load is simulated. The structure is fixed on one end. Quadratic 27-node hexahedral finite elements are used for the discretization of the domain. The well-known St. Venant-Kirchhoff material is used as constitutive law, which is a generalization of Hooke's law to the nonlinear realm.

Study the setup shown in fig. 1 and the comments in the input file `solid_cantilever.py`. Run the simulation, either in one of the provided Docker containers or using your own FEniCSx/Ambit installation, using the command

```
mpirun -n 1 python3 solid_cantilever.py
```

It is fully sufficient to use one core (`mpirun -n 1`) for the presented setup. Open the results file `results_solid_cantilever_displacement.xdmf` in Paraview, and visualize the deformation over time.

Figure 2 shows the displacement magnitude at the end of the simulation.

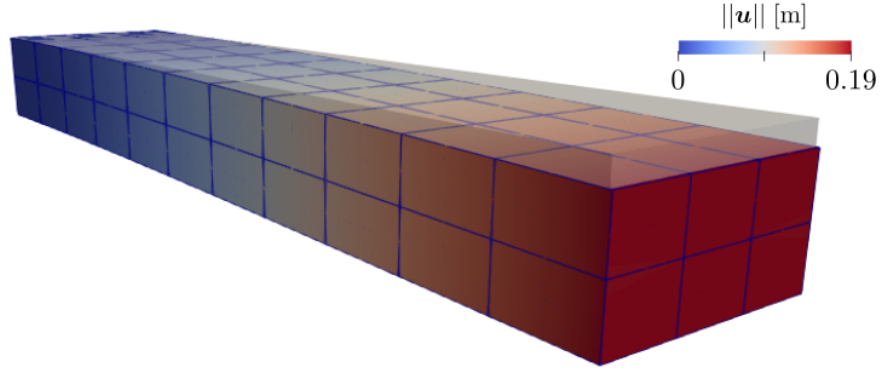


Figure 2: Cantilever, tip deformation. Color shows displacement magnitude.

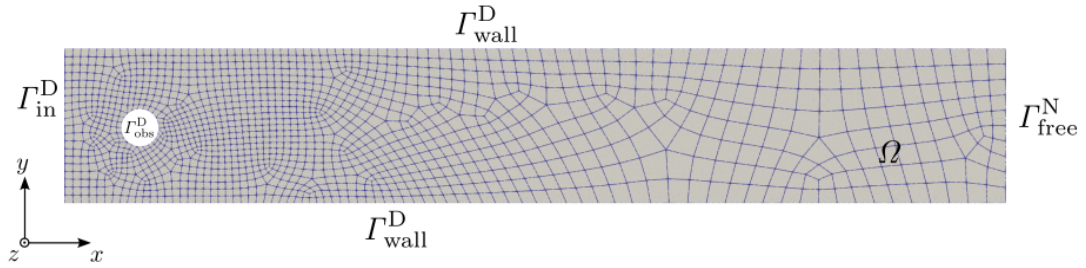


Figure 3: Channel flow, problem setup.

5.2 Demo: Fluid

- Physics description given in sec. 4.2
- Input files: `demos/fluid`

2D channel flow

This example shows how to set up 2D fluid flow in a channel around a rigid obstacle. Incompressible Navier-Stokes flow is solved using Taylor-Hood elements (9-node biquadratic quadrilaterals for the velocity, 4-node bilinear quadrilaterals for the pressure). Study the setup and the comments in the input file `fluid_channel.py`. Run the simulation, either in one of the provided Docker containers or using your own FEniCSx/Ambit installation, using the command

```
mpirun -n 1 python3 fluid_channel.py
```

It is fully sufficient to use one core (`mpiexec -n 1`) for the presented setup.

Open the results file `results_fluid_channel_velocity.xdmf` and

`results_fluid_channel_pressure.xdmf` in Paraview, and visualize the velocity as well as the pressure over time.

Fig. 4 shows the velocity magnitude (top) as well as the pressure (bottom part) at the end of the simulation.

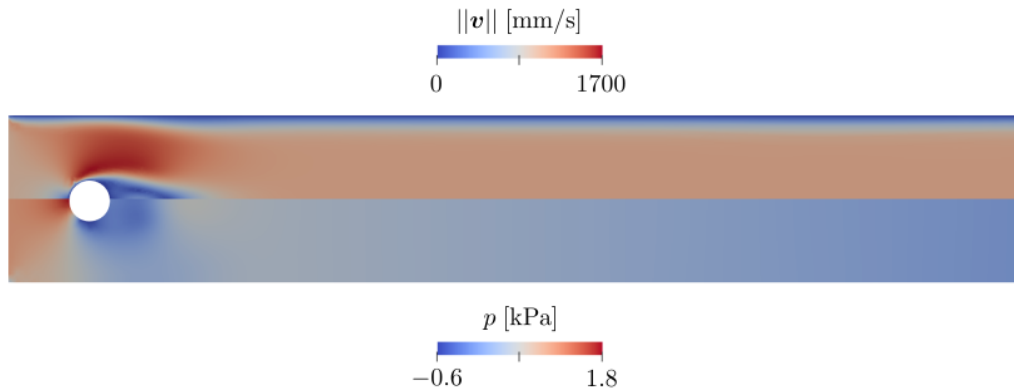


Figure 4: Velocity magnitude (top part) and pressure (bottom part) at end of simulation.

5.3 Demo: 0D flow

- Physics description given in sec. 4.3
- Input files: `demos/flow0d`

Systemic and pulmonary circulation

This example demonstrates how to simulate a cardiac cycle using a lumped-parameter (0D) model for the heart chambers and the entire circulation. Multiple heart beats are run until a periodic state criterion is met (which compares variable values at the beginning to those at the end of a cycle, and stops if the relative change is less than a specified value, here `'eps_periodic'` in the `TIME_PARAMS` dictionary). The problem is set up such that periodicity is reached after 5 heart cycles.

Study the setup in fig. 5 and the comments in the input file `flow0d_heart_cycle.py`. Run the simulation, either in one of the provided Docker containers or using your own FEniCSx/Ambit installation, using the command

```
python3 flow0d_heart_cycle.py
```

For postprocessing of the time courses of pressures, volumes, and fluxes of the 0D model, either use your own tools to plot the text output files (first column is time, second is the

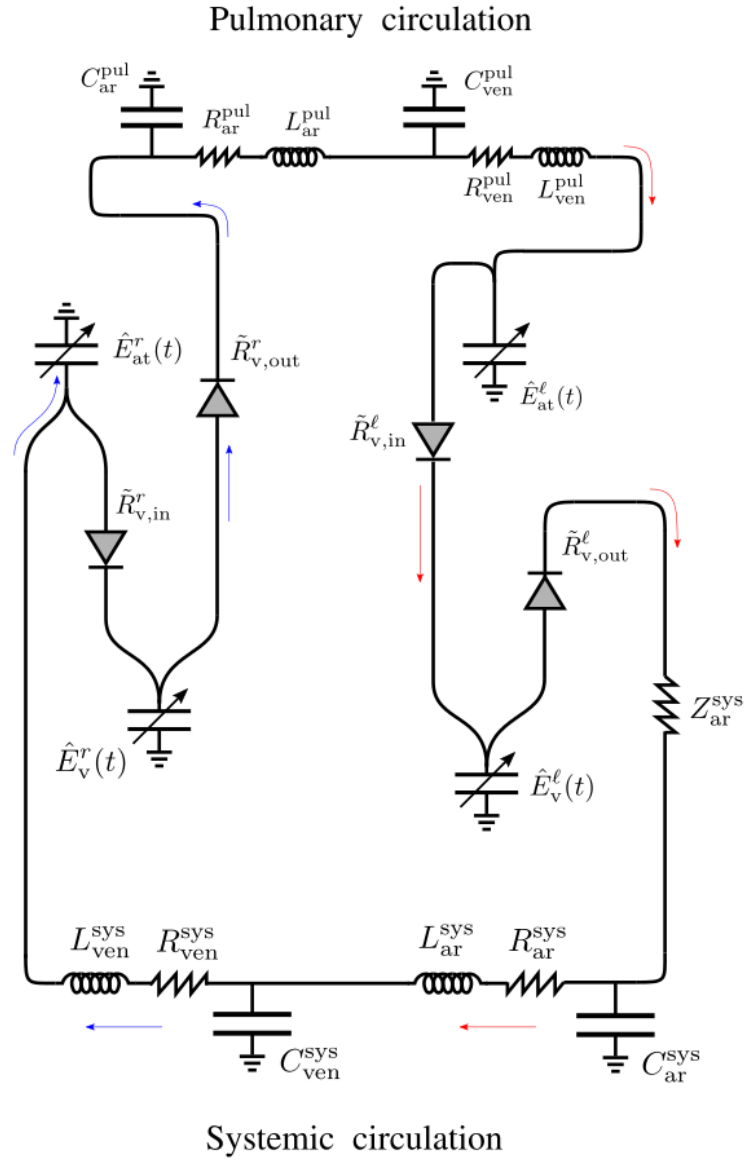


Figure 5: 0D heart, systemic and pulmonary circulation, problem setup.

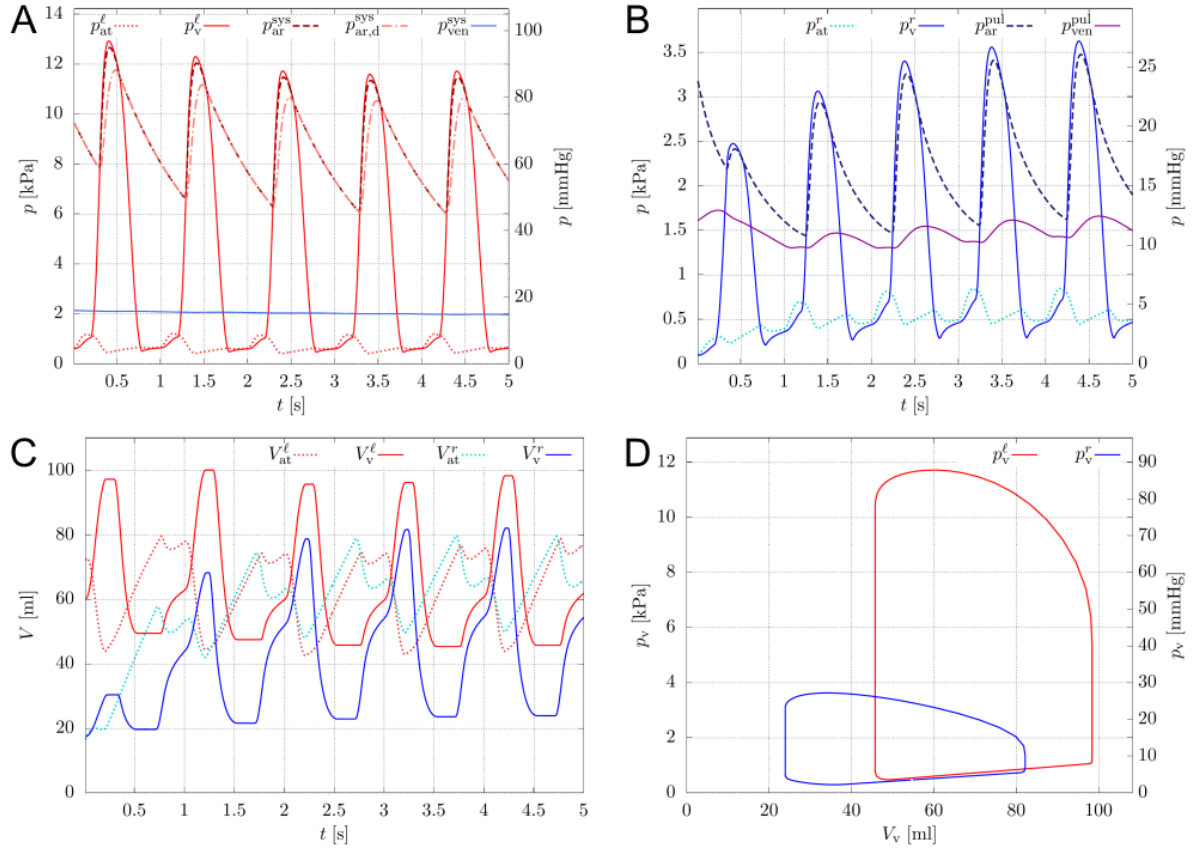


Figure 6: A. Left heart and systemic pressures over time. B. Right heart and pulmonary pressures over time. C. Left and right ventricular and atrial volumes over time. D. Left and right ventricular pressure-volume relationships of periodic (5th) cycle.

respective quantity), or make sure to have Gnuplot (and TeX) installed and navigate to the output folder (`tmp/`) in order to execute the script `flow0d_plot.py` (which lies in `ambit/src/ambit_fe/postprocess/`):

```
flow0d_plot.py -s flow0d_heart_cycle -n 100
```

A folder `plot_flow0d_heart_cycle` is created inside `tmp/`. Look at the results of pressures (p), volumes (V), and fluxes (q , Q) over time. Subscripts `v`, `at`, `ar`, `ven` refer to ‘ventricular’, ‘atrial’, ‘arterial’, and ‘venous’, respectively. Superscripts `l`, `r`, `sys`, `pul` refer to ‘left’, ‘right’, ‘systemic’, and ‘pulmonary’, respectively. Try to understand the time courses of the respective pressures, as well as the plots of ventricular pressure over volume. Check that the overall system volume is constant and around 4-5 liters.

The solution is depicted in fig. 6, showing the time course of volumes and pressures of the circulatory system.

5.4 Demo: Solid + 0D flow

- Physics description given in sec. 4.4
- Input files: `demos/solid_flow0d`

3D heart, coupled to systemic and pulmonary circulation

This example demonstrates how to set up and simulate a two-chamber (left and right ventricular) solid mechanics heart model coupled to a closed-loop 0D circulatory system. A full dynamic heart cycle of duration 1 s is simulated, where the active contraction is modeled by a prescribed active stress approach. Passive material behavior of the heart muscle is governed by the Holzapfel-Ogden anisotropic strain energy function [15] and a strain rate-dependent viscous model [4]. We start the simulation with "prestressing" using the MULF method [6, 18], which allows to imprint loads without changing the geometry, where the solid is loaded to the initial left and right ventricular pressures. Thereafter, we kickstart the dynamic simulation with passive ventricular filling by the systole of the atria (0D chamber models). Ventricular systole happens in $t \in [0.2 \text{ s}, 0.53 \text{ s}]$, hence lasting a third of the whole cycle time. After systole, the heart relaxes and eventually fills to about the same pressure as it has been initialized to.

NOTE: For demonstrative purposes, a fairly coarse finite element discretization is chosen here, which by no means yields a spatially converged solution and which may be prone to locking phenomena. The user may increase the parameter ``order_disp'` in the `FEM_PARAMS` section from 1 to 2 (and increase ``quad_degree'` to 6) such that quadratic finite element ansatz functions (instead of linear ones) are used. While this will increase accuracy and mitigate locking, computation time will increase.

Study the setup shown in fig. 7 and the comments in the input file `solid_flow0d_heart_cycle.py`. Run the simulation, either in one of the provided Docker containers or using your own FEniCSx/Ambit installation, using the command

```
mpiexec -n 1 python3 solid_flow0d_heart_cycle.py
```

It is fully sufficient to use one core (`mpiexec -n 1`) for the presented setup, while you might want to use more (e.g., `mpiexec -n 4`) if you increase ``order_disp'` to 2.

Open the results file `results_solid_flow0d_heart_cycle_displacement.xdmf` in Paraview, and visualize the deformation over the heart cycle.

For postprocessing of the time courses of pressures, volumes, and fluxes of the 0D model, either use your own tools to plot the text output files (first column is time, second is the respective quantity), or make sure to have Gnuplot (and TeX) installed and navigate to the output folder (`tmp/`) in order to execute the script `flow0d_plot.py` (which lies in `ambit/src/ambit_fe/postprocess/`):

```
flow0d_plot.py -s solid_flow0d_heart_cycle -V0 117e3 93e3 0 0 0
```

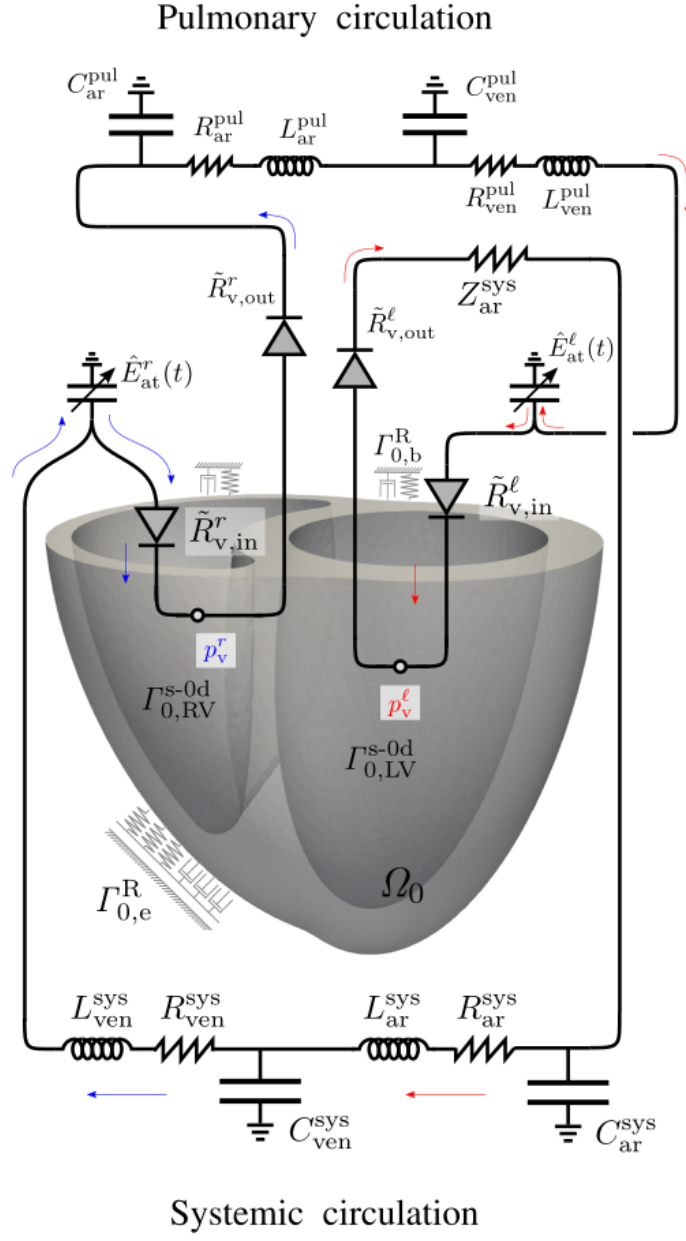


Figure 7: Generic 3D ventricular heart model coupled to a closed-loop systemic and pulmonary circulation model.

A folder `plot_solid_flow0d_heart_cycle` is created inside `tmp/`. Look at the results of pressures (p), volumes (V), and fluxes (q , Q) over time. Subscripts `v`, `at`, `ar`, `ven` refer to ‘ventricular’, ‘atrial’, ‘arterial’, and ‘venous’, respectively. Superscripts `l`, `r`, `sys`, `pul` refer to ‘left’, ‘right’, ‘systemic’, and ‘pulmonary’, respectively. Try to understand the time courses of the respective pressures, as well as the plots of ventricular pressure over volume. Check that the overall system volume is constant and around 4-5 liters.

NOTE: This setup computes only one cardiac cycle, which does not yield a periodic state solution (compare e.g. initial and end-cyclic right ventricular pressures and volumes, which do not coincide). Change the parameter `number_of_cycles` from 1 to 10 and re-run the simulation. The simulation will stop when the cycle error (relative change in 0D variable quantities from beginning to end of a cycle) falls below the value of ``eps_periodic'` (set to 5%). How many cycles are needed to reach periodicity?

Figure 8 shows a high-fidelity solution using a refined mesh and quadratic tetrahedral elements. Compare your solution from the coarser mesh. What is the deviation in ventricular volume?

5.5 Demo: Fluid + 0D flow

- Physics description given in sec. 4.4
- Input files: `demos/fluid_flow0d`

Blocked pipe flow with 0D model bypass

This example demonstrates how to couple 3D fluid flow to a 0D lumped-parameter model. Incompressible transient Navier-Stokes flow in a pipe with prescribed inflow is solved, with the special constraint that an internal boundary (all-time closed valve) separates region 1 and region 2 of the pipe. This internal Dirichlet condition can only be achieved by splitting the pressure space, hence having duplicate pressure nodes at the valve plane. Otherwise, fluid would experience deceleration towards the valve and unphysical acceleration behind it, since the pressure gradient drives fluid flow.

This example demonstrates how the closed valve can be bypassed by a 0D flow model that links the 3D fluid out-flow of one region to the in-flow of the other region. The 0D model consists of two Windkessel models in series, each having compliance, resistance, and inertance elements.

Study the setup shown in fig. 9 and the comments in the input file `fluid_flow0d_pipe.py`. Run the simulation, either in one of the provided Docker containers or using your own FEniCSx/Ambit installation, using the command

```
mpiexec -n 1 python3 fluid_flow0d_pipe.py
```

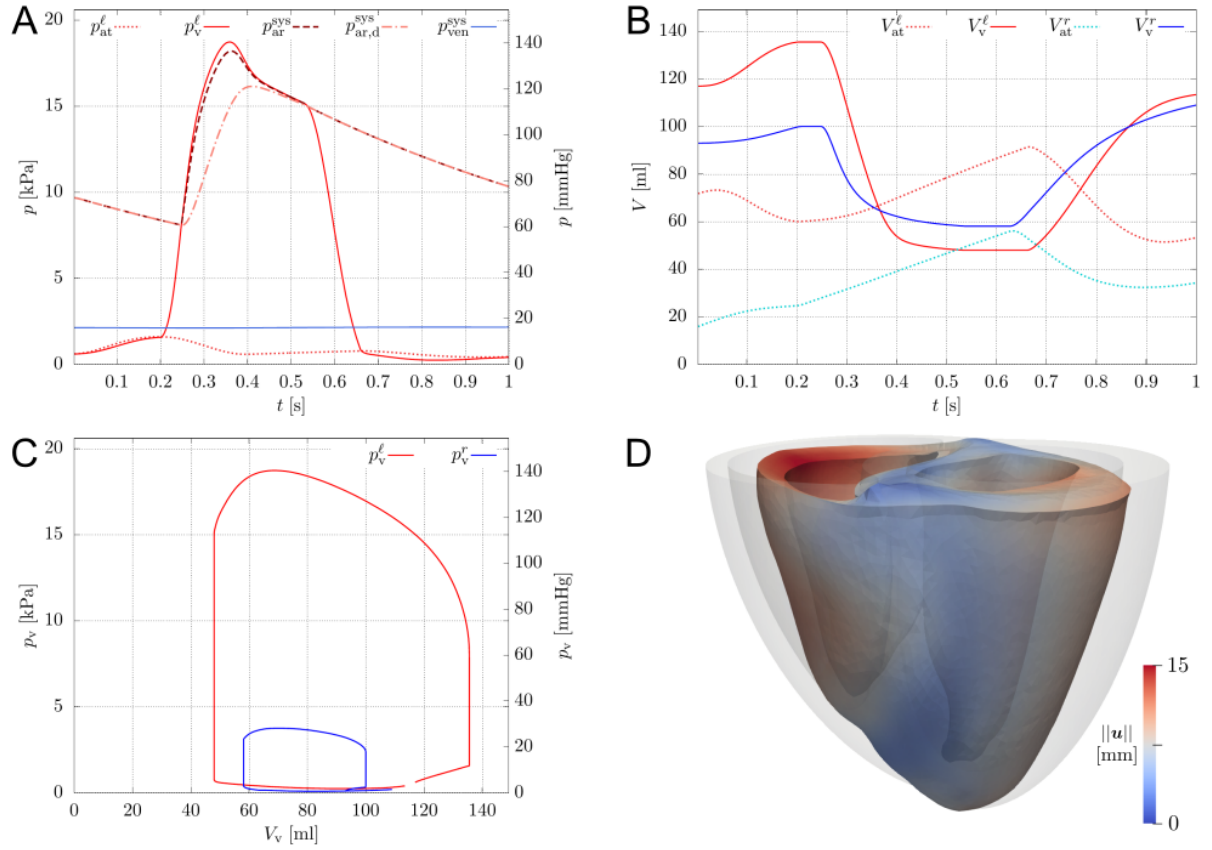



Figure 8: A. Left heart and systemic pressures over time. B. Left and right ventricular and atrial volumes over time. C. Left and right ventricular pressure-volume relationships. D. Snapshot of heart deformation at end-systole, color indicates displacement magnitude.

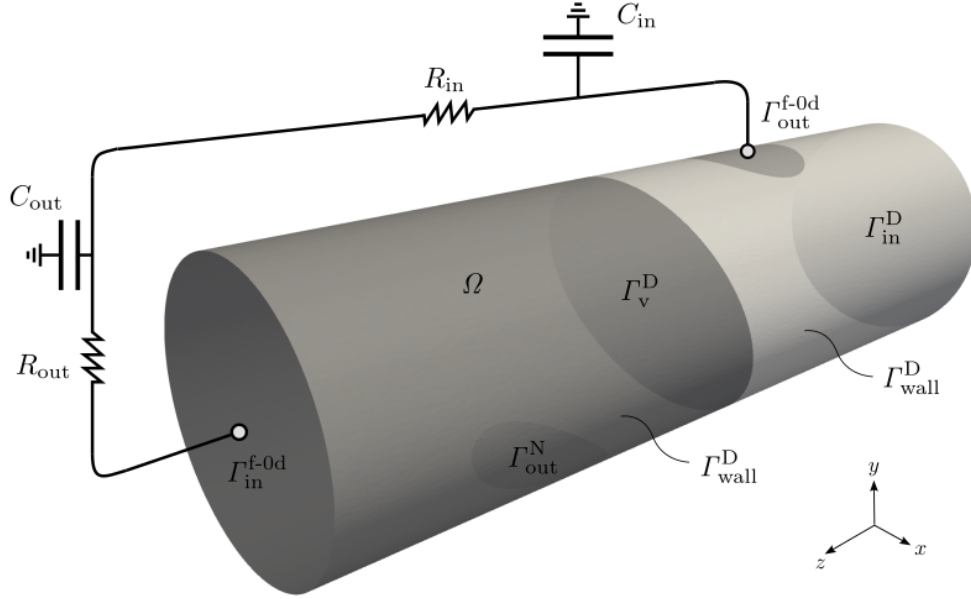


Figure 9: Blocked pipe with 0D model bypass, simulation setup.

It is fully sufficient to use one core (`mpirun -n 1`) for the presented setup.

Open the results file `results_fluid_flow0d_pipe_velocity.xdmf` in Paraview, and visualize the velocity over time.

Think of which parameter(s) of the 0D model to tweak in order to achieve a) little to no fluid in-flow (into Γ_{in}^{f-0d}), b) almost the same flow across Γ_{out}^{f-0d} and Γ_{in}^{f-0d} . Think of where the flow is going to in case of a).

Figure shows the velocity streamlines and magnitude at the end of the simulation.

5.6 Demo: FSI

- Physics description given in sec. 4.4
- Input files: `demos/fsi`

Channel flow around elastic flag

Incompressible fluid flow in a 2D channel around an elastic flag is studied. The setup corresponds to the well-known Turek benchmark [21]. Here, the two cases FSI2 and FSI3 from the original setup are investigated. A prescribed inflow velocity with parabolic inflow profile is used:

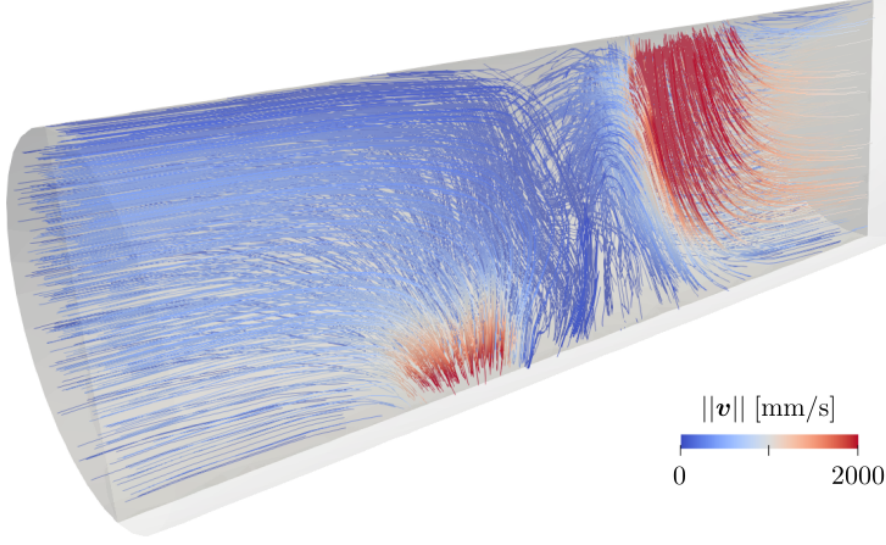


Figure 10: Streamlines of velocity at end of simulation, color indicates velocity magnitude.

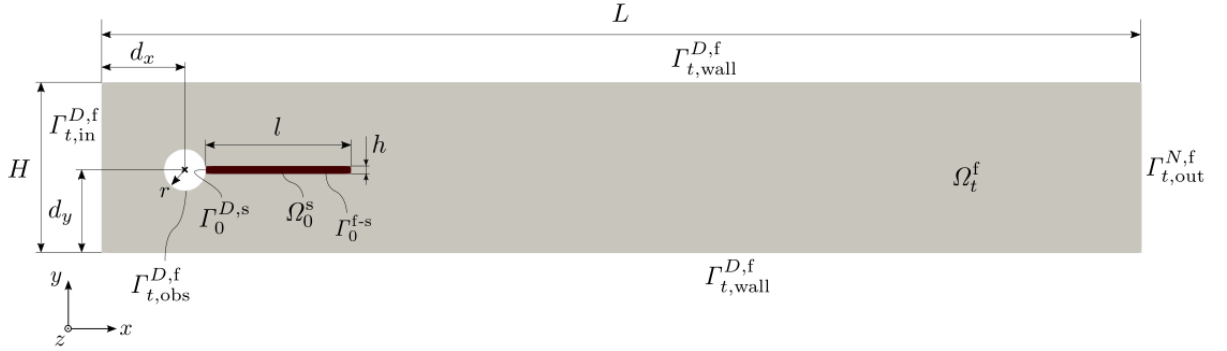


Figure 11: Channel flow around an elastic flag [21], problem setup.

$$\mathbf{v}_f = \bar{v}(t, y) \mathbf{e}_x \quad \text{on } \Gamma_{t,\text{in}}^{D,f} \times [0, T], \quad (121)$$

with

$$\bar{v}(t, y) = \begin{cases} 1.5 \bar{U} \frac{y(H-y)}{\left(\frac{H}{2}\right)^2} \frac{1 - \cos\left(\frac{\pi t}{2}\right)}{2}, & \text{if } t < 2, \\ 1.5 \bar{U} \frac{y(H-y)}{\left(\frac{H}{2}\right)^2}, & \text{else,} \end{cases} \quad (122)$$

with $\bar{U} = 10^3$ mm/s (FSI2) and $\bar{U} = 2 \cdot 10^3$ mm/s (FSI3).

Geometrical parameters, given in [mm], are:

The fluid is discretized with quadrilateral $\mathbb{Q}^2$ - $\mathbb{Q}^1$ Taylor-Hood finite elements, hence no stabilization is needed. The solid is discretized with displacement-based quadrilateral $\mathbb{Q}^2$

L	H	r	l	h	d_x	d_y
2500	410	50	350	20	200	200

elements. Temporal discretization for both the solid and the fluid are carried out with a Generalized- α scheme with no numerical damping ($\rho_{\text{inf}} = 1$).

Study the setup shown in fig. 11 together with the parameters in the table and the comments in the input file `fsi_channel_flag.py`. Run the simulation for FSI2 and FSI3 cases, either in one of the provided Docker containers or using your own FEniCSx/Ambit installation, using the command

```
mpiexec -n 1 python3 fsi_channel_flag.py
```

If your system allows, use more than one core (e.g. `mpiexec -n 4`) in order to speed up the simulation a bit.

The physics of the problem are strongly time-dependent, and a (near-)periodic oscillation of the flag only occurs after $t \approx 10$ s (FSI2) and $t \approx 5$ s (FSI3). Run the problem to the end ($t = 15$ s for FSI2, $t = 7.5$ s for FSI3), be patient, and monitor the flag tip displacement over time.

Figure 12 depicts the velocity at three instances in time towards the end of the simulation for the FSI2 case, and figure 13 shows the flag's tip displacement over time compared to the reference solution, over a time interval where the solution has become periodic. (Note that the reference solution from the official link shown in the input file needs to be time-adjusted, i.e. synchronized with the first peak in the interval of interest, since the time column of the reference data does not correspond to the physical time of the problem setup.)

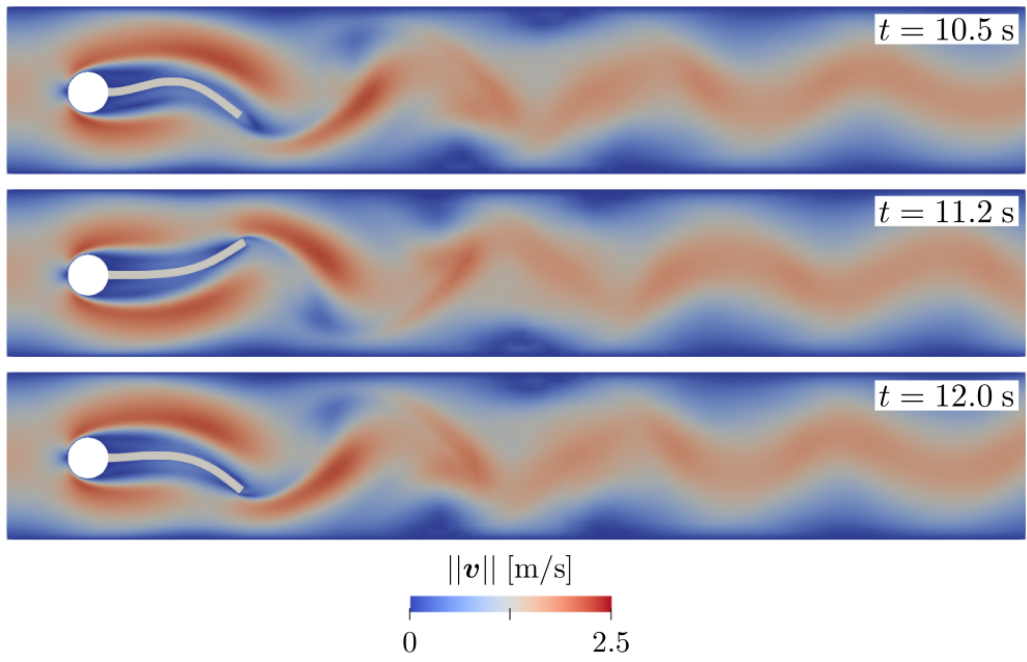


Figure 12: FSI2 case: Magnitude of fluid velocity at three instances in time ($t = 10.5 \text{ s}$, $t = 11.2 \text{ s}$, and $t = 12 \text{ s}$) towards end of simulation, color indicates velocity magnitude.

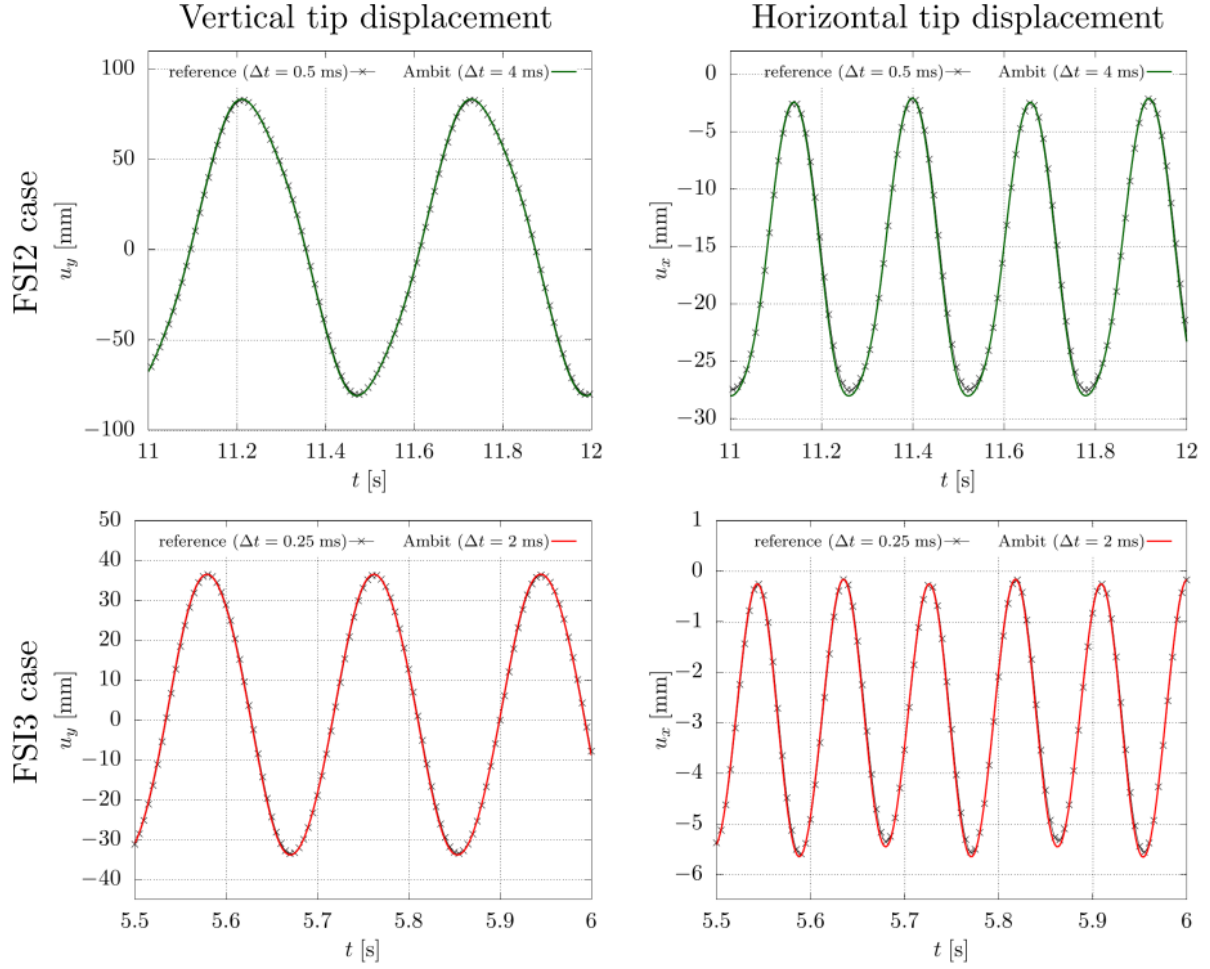


Figure 13: Comparison to benchmark reference solution for the time course of the flag's tip displacement for the two setups FSI2 and FSI3. A fairly coarse time step of $\Delta t = 4$ ms (FSI2) and $\Delta t = 2$ ms (FSI3) already allows a close match to the original results.

5.7 Demo: Multiphase fluid

- Physics description given in sec. 4.4
- Input files: `demos/fluid_multiphase`

Rising bubble under gravity

We study a 2D multiphase flow benchmark, where two incompressible fluids of different densities and different viscosities interact: a fluid of lower density (and viscosity) is immersed into a surrounding fluid of higher density (and viscosity), and a gravitational force acts in vertical direction, cf. the setup in Fig. 14. This setup corresponds to the benchmark case described in [3],[19]. The following Dirichlet boundary conditions apply:

$$\begin{aligned} \mathbf{v} &= \mathbf{0} & \text{on } \Gamma_b^D \cup \Gamma_t^D \times [0, T], \\ \mathbf{v} \cdot \mathbf{n} &= 0 & \text{on } \Gamma_r^D \cup \Gamma_l^D \times [0, T]. \end{aligned} \quad (123)$$

The body force term takes the form $\hat{\mathbf{b}} = -\rho \mathbf{g}$, with $\mathbf{g} \cdot \mathbf{e}_y = 0.98$. Our phase field variable is bounded between $a = -1$ and $b = 1$. Further, Cahn-Hilliard parameters are chosen as

$$D_0 = \frac{\tilde{\sigma}}{4\epsilon}, \quad M_0 = 0.1\epsilon^2, \quad \kappa = \tilde{\sigma}\epsilon, \quad (124)$$

with $\tilde{\sigma} = \frac{3\sigma}{2\sqrt{2}}$ (surface energy density coefficient) and $\epsilon = 0.64 h_e$, where h_e is the finite element edge length. The mobility exponent in Eq. (117) is chosen to $\gamma = 1$. Parameters for the two cases are shown in the following table. Bulk viscosities are assumed zero ($\zeta = 0$).

	ρ_1	ρ_2	η_1	η_2	σ
Case 1	1000	100	10	1	24.5
Case 2	1000	1	10	0.1	1.96

The fluid is discretized with quadrilateral $\mathbb{Q}^2$ - $\mathbb{Q}^1$ Taylor-Hood finite elements, while a quadrilateral $\mathbb{Q}^1$ - $\mathbb{Q}^1$ discretization is chosen for the Cahn-Hilliard equations. A BDF2 time-integration scheme is chosen for both fluid and Cahn-Hilliard.

Study the setup shown in Fig. 14 together with the parameters in the table and the comments in the input file `fluid_multiphase_rising_bubble.py`. Run the simulation for both cases 1 and 2, either in one of the provided Docker containers or using your own FEniCSx/Ambit installation, using the command

```
mpirun -n 1 python3 fluid_multiphase_rising_bubble.py
```

Try different settings for the mesh, e.g. vary `meshsize` inside `mesh_domain` (increase from `[32,64]` to `[64,128]` to `[128,256]`; the first value is the number of elements used in

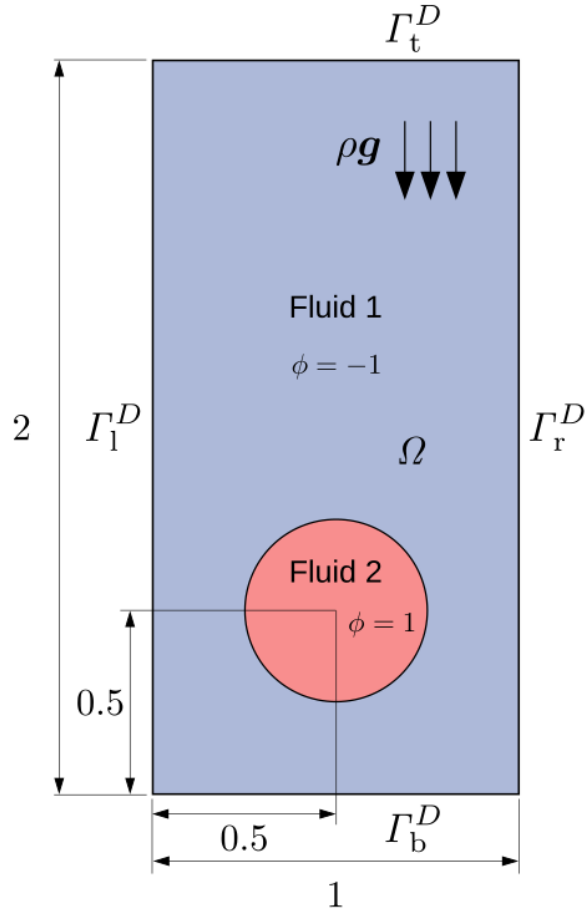


Figure 14: Rising bubble benchmark [3],[19]: A fluid with lower density (fluid 2) is surrounded by a fluid of higher density (fluid 1). Bottom and top velocities are constrained, as well as the normal velocity on the left and right walls. A gravitational force acts in vertical direction.

horizontal, the second the number in vertical direction). If your system allows, use more than one core, especially for finer mesh sizes (hence run with `mpiexec -n 4 ...` or more). A good rule of thumb is to have 10000–20000 degrees of freedom per process. The time step and the interface thickness parameter ϵ will be changed automatically depending on the chosen mesh size. Compare the results for the two cases and the different mesh sizes.

Figure 15 shows the results of phase field and magnitude of fluid velocity for a mesh size of 128×256 for both cases at the end of the simulation time $t = 3$.

Figure 16 plots the center of mass as well as the rise velocity of the lower-density fluid bubble over time (mesh size 128×256) and compares the solution with two different reference solutions (one a mass-, the other a volume-averaged velocity formulation) from Brunk and ten Eikelder [3] as well as ten Eikelder and Schillinger [19]. We observe good agreements between our (mass-averaged velocity) solution and the volume-averaged velocity reference solutions. Slight deviations to the mass-averaged reference solutions are observed.

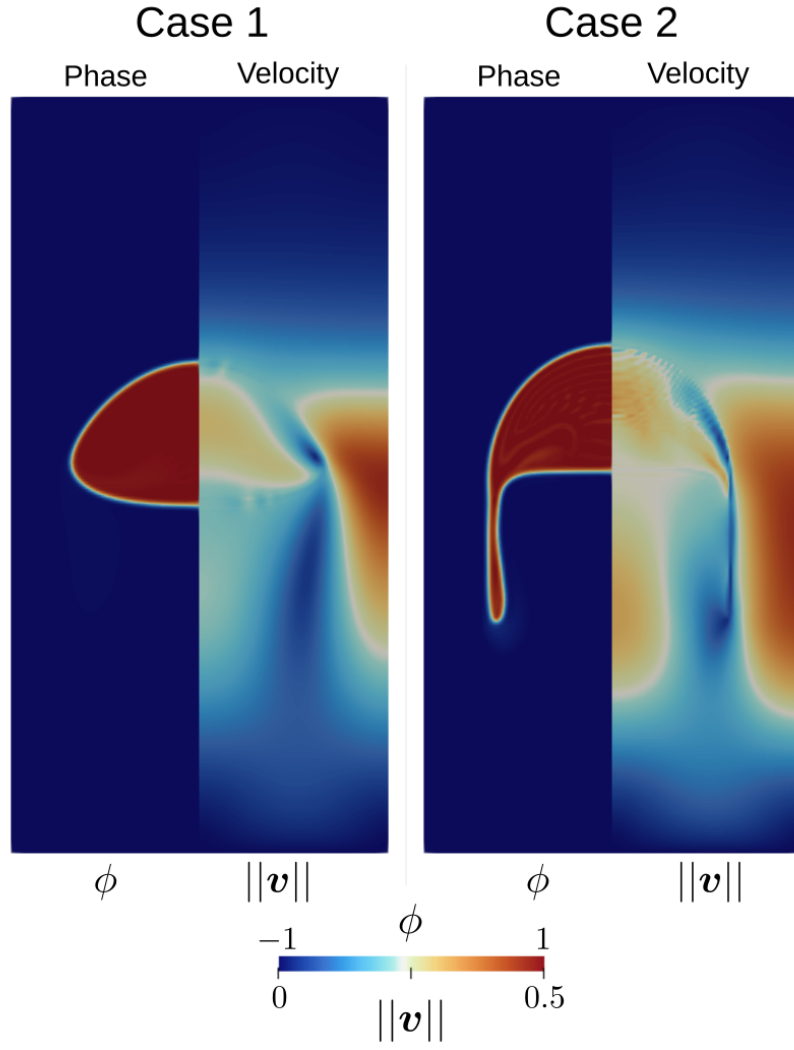


Figure 15: Phase field ϕ (left part) and magnitude of velocity $||\mathbf{v}||$ (right part) solutions for both cases at the final time $t = 3.0$.

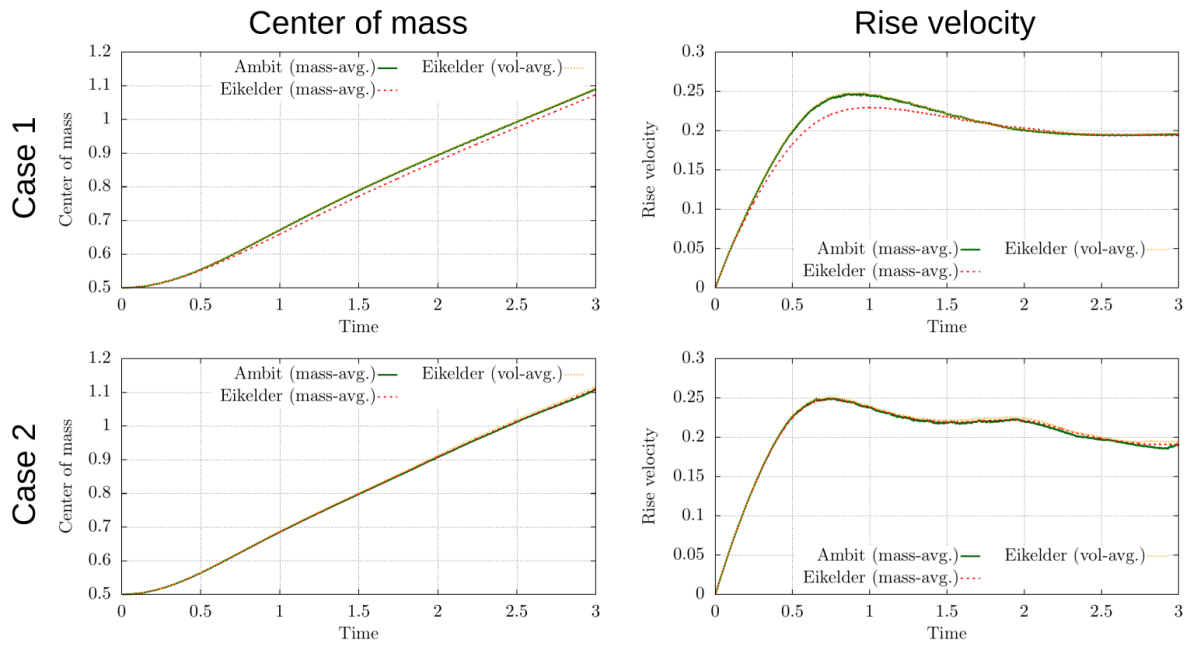


Figure 16: Center of mass (left column) and rise velocity (right column) for both cases over time, comparison to reference solutions from Brunk and ten Eikelder [3].

6 Table of symbols

Ω_0, Ω	: reference, current domain
Γ_0, Γ	: reference, current boundary
$\mathbf{x}_0, \mathbf{x}$	: coordinates of the reference, current frame
$\mathbf{e}_x, \mathbf{e}_y, \mathbf{e}_z$	: unit vectors of the cartesian reference frame
$\mathbf{n}_0, \mathbf{n}$	: unit outward normal defined in the reference, current frame
∇_0, ∇	: Nabla operator with respect to the reference, current frame
$\nabla \mathbf{v} := \frac{\partial v_i}{\partial x_j} \mathbf{e}_i \otimes \mathbf{e}_j$	: Gradient of a vector field
$\nabla \cdot \mathbf{v} := \frac{\partial v_i}{\partial x_j} \mathbf{e}_i \cdot \mathbf{e}_j$	: Divergence of a vector field
$\mathbf{e}_i$	: Basis vectors, $i \in \{1, 2, 3\}$
t, T	: current, end time of an initial (boundary) value problem
$\mathbf{u}, \hat{\mathbf{u}}_0$	: solid mechanics displacement field, and prescribed initial value
$\delta \mathbf{u}, \Delta \mathbf{u}$	: solid mechanics displacement test, trial function
p	: solid mechanics hydrostatic pressure, or fluid mechanics pressure
$\delta p, \Delta p$	: solid or fluid mechanics pressure test, trial function
$\mathbf{v} = \frac{\partial \mathbf{u}}{\partial t}, \hat{\mathbf{v}}_0$	: solid mechanics velocity, and prescribed initial value
$\mathbf{a} = \frac{\partial^2 \mathbf{u}}{\partial t^2}$	: solid mechanics acceleration
$\mathbf{v}, \hat{\mathbf{v}}_0$	: fluid mechanics velocity, and prescribed initial value
$\delta \mathbf{v}, \Delta \mathbf{v}$	: fluid mechanics velocity test, trial function
$\mathbf{a} = \frac{\partial \mathbf{v}}{\partial t}$	: fluid mechanics acceleration
$\mathbf{d}$	: ALE domain displacement
$\delta \mathbf{d}, \Delta \mathbf{d}$	: ALE domain displacement test, trial function
$\hat{\mathbf{b}}_0, \hat{\mathbf{b}}$	: body force vector defined in the reference, current frame
$\hat{\mathbf{w}} = \frac{\partial \mathbf{d}}{\partial t}$	: ALE domain velocity
ρ_0, ρ	: reference, current density
ϕ, μ	: phase field, potential in Cahn-Hilliard equations

$\boldsymbol{P} = \boldsymbol{F}\boldsymbol{S}$	: 1st Piola Kirchhoff stress tensor
$\boldsymbol{F} = \boldsymbol{I} + \nabla_0 \boldsymbol{u}$	: solid deformation gradient
$\hat{\boldsymbol{F}} = \boldsymbol{I} + \nabla_0 \boldsymbol{d}$	: ALE deformation gradient
$J = \det \boldsymbol{F}$	: determinant of solid deformation gradient
$\hat{J} = \det \hat{\boldsymbol{F}}$	: determinant of ALE deformation gradient
$\boldsymbol{S}$	: 2nd Piola-Kirchhoff stress tensor
$\boldsymbol{\sigma}$	: Cauchy stress tensor
$\boldsymbol{t}_0, \hat{\boldsymbol{t}}_0$	: 1st Piola-Kirchhoff traction, prescribed 1st Piola-Kirchhoff traction
$\boldsymbol{t}, \hat{\boldsymbol{t}}$	: Cauchy traction, prescribed Cauchy traction

Bibliography

- [1] C. J. Arthurs, K. D. Lau, K. N. Asrress, S. R. Redwood, and C. A. Figueroa. A mathematical model of coronary blood flow control: simulation of patient-specific three-dimensional hemodynamics during exercise. *Am J Physiol Heart Circ Physiol*, 310(9):H1242–H1258, 2016.
- [2] S. Balay, S. Abhyankar, M. F. Adams, S. Benson, J. Brown, P. Brune, K. Buschelman, E. Constantinescu, L. Dalcin, A. Dener, V. Eijkhout, W. D. Gropp, V. Hapla, T. Isaac, P. Jolivet, D. Karpeev, D. Kaushik, M. G. Knepley, F. Kong, S. Kruger, D. A. May, L. C. McInnes, R. T. Mills, L. Mitchell, T. Munson, J. E. Roman, K. Rupp, P. Sanan, J. Sarich, B. F. Smith, S. Zampini, H. Zhang, H. Zhang, and J. Zhang. PETSc/TAO users manual. Technical Report ANL-21/39 - Revision 3.17, Argonne National Laboratory, 2022.
- [3] A. Brunk and M. F. P. ten Eikelder. A simple, fully-discrete, unconditionally energy-stable method for the two-phase Navier-Stokes Cahn-Hilliard model with arbitrary density ratios. *Journal of Computational Physics*, 548:114558, 2026.
- [4] D. Chapelle, P. L. Tallec, P. Moireau, and M. Sorine. Energy-preserving muscle tissue model: formulation and compatible discretizations. *Journal for Multiscale Computational Engineering*, 10(2):189–211, 2012.
- [5] C. Farhat, P. Avery, T. Chapman, and J. Cortial. Dimensional reduction of nonlinear finite element dynamic models with finite rotations and energy-based mesh sampling and weighting for computational efficiency. *International Journal for Numerical Methods in Engineering*, 98(9):625–662, 2014.
- [6] M. W. Gee, C. Förster, and W. A. Wall. A computational strategy for prestressing patient-specific biomechanical problems under finite deformation. *International Journal for Numerical Methods in Biomedical Engineering*, 26(1):52–72, 2010.
- [7] M. W. Gee, C. T. Kelley, and R. B. Lehoucq. Pseudo-transient continuation for nonlinear transient elasticity. *International Journal for Numerical Methods in Engineering*, 78(10):1209–1219, 2009.

- [8] S. Göktepe, O. J. Abilez, K. K. Parker, and E. Kuhl. A multiscale model for eccentric and concentric cardiac growth through sarcomerogenesis. *J Theor Biol*, 265(3):433–442, 2010.
- [9] J. M. Guccione, K. D. Costa, and A. D. McCulloch. Finite element stress analysis of left ventricular mechanics in the beating dog heart. *J Biomech*, 28(10):1167–1177, 1995.
- [10] M. Hirschvogel. *Computational modeling of patient-specific cardiac mechanics with model reduction-based parameter estimation and applications to novel heart assist technologies*. Verlag Dr. Hut, MediaTUM, 1 edition, 2019.
- [11] M. Hirschvogel. Ambit – A FEniCS-based cardiovascular multi-physics solver. *Journal of Open Source Software*, 9(93):5744, 2024.
- [12] M. Hirschvogel, M. Balmus, M. Bonini, and D. Nordsletten. Fluid-reduced-solid interaction (FrSI): Physics- and projection-based model reduction for cardiovascular applications. *Journal of Computational Physics*, 506:112921, 2024.
- [13] M. Hirschvogel, M. Bassilious, L. Jagschies, S. M. Wildhirt, and M. W. Gee. A monolithic 3D-0D coupled closed-loop model of the heart and the vascular system: Experiment-based parameter estimation for patient-specific cardiac mechanics. *Int J Numer Method Biomed Eng*, 33(8):e2842, 2017.
- [14] G. A. Holzapfel. *Nonlinear Solid Mechanics – A Continuum Approach for Engineering*. Wiley Press Chichester, 2000.
- [15] G. A. Holzapfel and R. W. Ogden. Constitutive modelling of passive myocardium: A structurally based framework for material characterization. *Phil Trans R Soc A*, 367(1902):3445–3475, 2009.
- [16] A. Logg, K.-A. Mardal, and G. N. Wells, editors. *Automated Solution of Differential Equations by the Finite Element Method – The FEniCS Book*. Springer, 2012.
- [17] D. A. Nordsletten, M. McCormick, P. J. Kilner, P. Hunter, D. Kay, and N. P. Smith. Fluid-solid coupling for the investigation of diastolic and systolic human left ventricular function. *International Journal for Numerical Methods in Biomedical Engineering*, 27(7):1017–1039, 2011.
- [18] A. Schein and M. W. Gee. Greedy maximin distance sampling based model order reduction of prestressed and parametrized abdominal aortic aneurysms. *Advanced Modeling and Simulation in Engineering Sciences*, 8(18), 2021.
- [19] M. F. P. ten Eikelder and D. Schillinger. The divergence-free velocity formulation of the consistent Navier-Stokes Cahn-Hilliard model with non-matching densities, divergence-conforming discretization, and benchmarks. *Journal of Computational Physics*, 513:113148, 2024.

- [20] T. E. Tezduyar and Y. Osawa. Finite element stabilization parameters computed from element matrices and vectors. *Computer Methods in Applied Mechanics and Engineering*, 190(3–4):411–430, 2000.
- [21] S. Turek and J. Hron. Proposal for numerical benchmarking of fluid-structure interaction between an elastic object and laminar incompressible flow. In H.-J. Bungartz and M. Schäfer, editors, *Lecture Notes in Computational Science and Engineering*, volume 53, pages 371–385. Springer Berlin Heidelberg, 2006.
- [22] N. Westerhof, J.-W. Lankhaar, and B. E. Westerhof. The arterial Windkessel. *Med Biol Eng Comput*, 47(2):H81–H88, 2009.